A Systems Perspective on Collective Communication for Large-Scale AI Training and Inference

KONG, Haoran

The Chinese University of Hong Kong, Shenzhen

May 2026

Abstract

Collective communication has become a central systems bottleneck in large-scale AI training and inference. As large language models scale from single-node training to clusters with tens of thousands or even more than one hundred thousand GPUs, the communication layer must support high throughput, low latency, topology awareness, fault tolerance, and workload-specific customization. Traditional collective communication libraries such as NCCL provide a strong foundation for common operations including AllReduce, AllGather, ReduceScatter, Broadcast, and AllToAll. However, modern workloads introduce new requirements: training needs scalable initialization and high bandwidth; inference needs low latency and low CPU overhead; mixture-of-experts models require dynamic and fine-grained all-to-all communication; and heterogeneous GPU clusters require communication schedules that adapt to topology and message size.

This thesis-style report reviews three representative systems: NCCLX from Meta, MSCCL++ from Microsoft, and SyCCL from Alibaba Cloud. NCCLX represents a production-oriented communication framework designed for the full training and inference lifecycle at 100K+ GPU scale. MSCCL++ rethinks GPU communication abstractions by separating low-level primitives, a domain-specific language, and NCCL-compatible collective APIs. SyCCL addresses the schedule-synthesis problem by exploiting symmetry in collective demands and cluster topologies. Together, these systems show that future collective communication libraries are evolving from fixed algorithm libraries into programmable, topology-aware, workload-aware, and operationally robust systems software.

The main argument of this report is that collective communication should be understood as a systems stack rather than a single library call. The three papers repeatedly point to the same pressure: applications need the stability of a mature communication library, but they also need the ability to specialize for new hardware, new model parallelism strategies, and new inference patterns. This report therefore treats NCCLX, MSCCL++, and SyCCL as complementary answers to one broader question: how should the communication layer be redesigned when model scale, cluster scale, and workload diversity all increase at the same time?

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

The rapid progress of large language models has changed the role of GPU communication in distributed machine learning systems. In earlier deep learning workloads, communication was often treated as a secondary cost around gradient synchronization. In modern LLM systems, communication is on the critical path of both training and inference. Training pipelines combine data parallelism, tensor parallelism, pipeline parallelism, expert parallelism, and sharded data parallelism. Inference services, especially those using mixture-of-experts models, require low-latency communication for token routing and expert aggregation.

Collective communication libraries such as NCCL have become the de facto foundation for GPU communication. They provide highly optimized implementations of standard collectives and hide many hardware details from application developers [6]. However, the fixed-algorithm and host-driven style of traditional collective libraries is increasingly stressed by new AI workloads. Small messages are dominated by latency and CPU overhead. Large messages are dominated by bandwidth and congestion. Dynamic MoE routing requires communication patterns that change from one forward pass to the next. Large clusters also introduce practical concerns such as startup time, fault localization, elasticity, and resource management.

These concerns appear together because LLM systems amplify both the data path and the control path. The data path moves tensors, gradients, activations, expert inputs, and expert outputs. The control path creates process groups, discovers topology, registers memory, exchanges addresses, tracks failures, and selects algorithms. In a small experiment, the control path may be ignored because it runs only once and affects only a few nodes. In a 100K-GPU training run, however, the control path becomes a systems bottleneck: initialization can take minutes, a single failed host can stall many parallelism groups, and repeated restarts can waste large amounts of accelerator time. This is why the NCCLX paper treats scalable initialization and fault analysis as first-class communication-library features rather than peripheral operational concerns [10].

The same pressure appears at the algorithmic level. Classical collectives such as ring AllReduce and tree Broadcast are still essential, but their assumptions are increasingly incomplete. Ring algorithms are bandwidth-efficient for large homogeneous transfers, yet they can be poor for small messages because each step adds latency. Tree algorithms reduce step count, but may underuse bandwidth on dense GPU fabrics. Hierarchical algorithms exploit node boundaries, but the best hierarchy depends on the ratio between intra-node and inter-node bandwidth. These trade-offs have long been studied in MPI collective communication [11]; the new difficulty is that modern GPU clusters add nonuniform links, GPU memory registration, CPU proxy threads, CUDA graph capture, and dynamic model-side routing.

This report studies three representative systems. NCCLX from Meta addresses the full production lifecycle of communication for 100K+ GPU training and inference [10]. MSCCL++ from Microsoft rethinks the programming interface of GPU communication by exposing primitive, DSL, and collective APIs [9]. SyCCL from Alibaba Cloud synthesizes topology-aware collective schedules by exploiting symmetry in the topology and collective demand [1]. Together, they

motivate a systems perspective: collective communication is not one library call, but a stack of abstractions, workloads, schedules, and runtime mechanisms.

The motivation for choosing these three papers is that they do not merely optimize the same baseline in slightly different ways. They attack three different failure modes of the traditional communication stack. NCCLX addresses the failure mode that a fast collective library is not enough when the system must initialize, recover, and debug jobs at 100K+ GPU scale. MSCCL++ addresses the failure mode that a convenient high-level API becomes a bottleneck when applications need custom, hardware-aware algorithms. SyCCL addresses the failure mode that fixed schedules waste bandwidth or incur unnecessary latency when the topology and collective demand differ from the assumptions of ring or tree algorithms. The three papers therefore cover a full path from application expression to schedule generation to production execution.

This perspective is especially important for LLM systems because the same model family can exercise multiple communication regimes. During training, the system may run large, synchronized collectives for gradient or parameter exchange. During prefill, it may require high throughput for medium-to-large activations. During decoding, it may require extremely low-latency small-message communication. During MoE routing, the communication volume and destination ranks may be determined by GPU-side routing decisions. A communication framework designed for only one of these regimes will leave performance or reliability on the table.

The model-parallelism literature makes this point concrete. Megatron-LM popularized tensor parallelism for transformer layers, which creates repeated AllReduce or ReduceScatter/AllGather communication inside the model step [5]. ZeRO and related optimizer-state sharding methods reduce memory pressure by distributing optimizer state, gradients, and parameters, but they also introduce structured communication during forward, backward, and optimizer phases [7]. MoE systems such as GShard and Switch Transformer reduce per-token computation by routing tokens to a subset of experts, but they make AllToAll-style exchange central to both training and serving [4, 2]. These systems differ in model architecture, yet they all turn collective communication into a design constraint rather than an implementation detail.

1.2 Three Research Questions

To structure this investigation, I organize the report around three questions that any practical large-scale AI communication stack must answer:

1. How do we program collective communication? What abstractions allow developers to express standard collectives, custom algorithms, one-sided transfers, and GPU-resident metadata without rewriting the entire stack?

2. What does the workload look like? What are the training, inference, topology, message-size, and evaluation requirements that determine whether a communication system is useful?

3. How do we run collective communication efficiently? How can a runtime or synthesizer exploit topology, symmetry, transport mechanisms, and operational feedback to reduce latency and improve bandwidth?

The three selected papers map naturally to these questions. MSCCL++ is primarily about programming abstractions. SyCCL is primarily about workload-specific schedule synthesis and evaluation. NCCLX is primarily about runtime and production deployment, including transport, initialization, inference customization, and failure analysis.

The first question is about the developer-facing surface. If developers only have access to standard collectives, they can express common training patterns but may struggle with dynamic or fused communication. If they have only low-level RDMA or memory operations, they gain control but lose portability and ease of use. MSCCL++ answers this by placing several interfaces in one stack: primitives for experts, a DSL for communication algorithms, and collective APIs for compatibility.

This question also includes the problem of who writes communication code. In a mature stack, ordinary model developers should not need to reason about queue pairs, memory fences,

or peer-access permissions. Communication-library experts, however, must be able to express nonstandard algorithms before they become common enough to be built into a fixed library. Hardware vendors must be able to expose new transport features without forcing every application to change. A good programming model therefore separates roles: application code expresses intent, algorithm code describes a schedule, backend code implements primitives, and runtime code handles resources. MSCCL++ is valuable because it explicitly recognizes these different users of the communication layer [9].

The second question is about the system-facing workload. Communication performance cannot be judged by one operation at one size. SyCCL shows that schedule quality changes with topology, message size, and collective type. NCCLX shows that production usefulness includes initialization, resource use, fault localization, and end-to-end model latency. This report therefore treats workloads as multi-dimensional rather than as isolated microbenchmarks.

The third question is about the runtime-facing implementation. Once an operation is expressed, the system must decide how to move data. NCCLX uses CTran, tensor registration, DQPLB, zero-copy transfer, and GPU-resident metadata. SyCCL searches sketches, synthesizes sub-schedules, and simulates candidates. MSCCL++ executes primitives through channels and DSL instructions. These mechanisms form the runtime side of the same design space.

The runtime question is also where reliability enters the thesis. A communication algorithm can be correct in isolation but unusable in a production cluster if it consumes too many queue pairs, registers memory too often, hides the true source of a hang, or cannot coexist with graph-captured inference execution. Large-scale AI systems therefore need runtimes that are performance-aware and failure-aware at the same time. NCCLX's CollTrace and Fault Analyzer are examples of this broader runtime responsibility [10]; they extend the communication layer from a fast path into an observable subsystem.

1.3 The Collective Communication Systems Stack

This report mirrors the layered structure of the reference thesis but replaces its agentic systems stack with a collective communication systems stack. At the programming layer, applications express communication through collectives, point-to-point operations, remote memory access, or lower-level primitives. At the workload and evaluation layer, systems are judged by training step time, inference latency, schedule performance, synthesis time, and hardware portability. At the runtime layer, the system chooses transports, manages memory registration, schedules data movement, initializes communicators, and localizes failures.

The stack perspective in Figure 1.1 is useful because none of the three selected papers solves the whole problem alone. A good primitive interface does not solve production initialization. A good production transport does not guarantee an optimal schedule. A good schedule synthesizer still needs an execution substrate. The central claim of this report is that future collective communication systems will combine all three layers.

Figure 1.2 provides a concrete production example of the stack view. The figure is important because NCCLX is not presented as only a replacement for one NCCL kernel. It is a layered communication framework that includes PyTorch integration, NCCL compatibility, CTran transport mechanisms, custom collective support, and operational tooling. In the later chapters, the same figure serves as the visual anchor for the argument that production collective communication includes both the fast path for bytes and the control path for metadata, initialization, and diagnosis.

1.4 Thesis Contributions

This thesis-style report makes four contributions. First, it summarizes the communication bottlenecks introduced by modern LLM training and inference. Second, it presents MSCCL++ as a programming-model contribution for GPU communication. Third, it treats NCCLX and

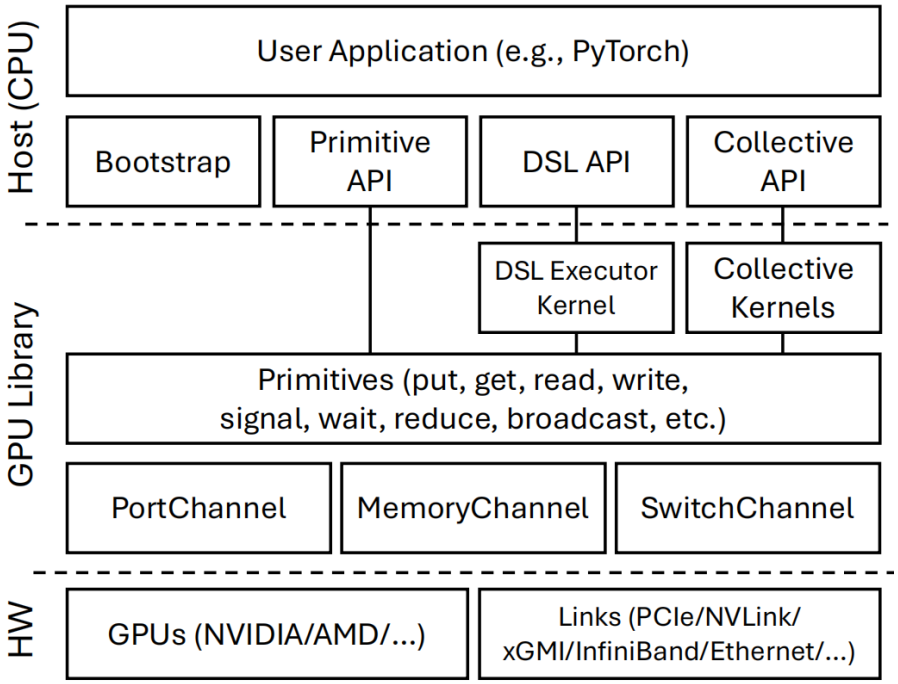


Figure 1.1: The three-layer collective communication systems stack.

SyCCL as complementary workload, evaluation, runtime, and scheduling contributions. Fourth, it synthesizes the three papers into a cross-layer view of future communication systems.

The report is deliberately written as a systems thesis rather than as a catalogue of optimizations. This matters because the same mechanism can have different meanings at different layers. For example, zero-copy transfer is a transport optimization, but it is also a memory-lifetime problem and an evaluation problem because it changes GPU memory pressure and SM usage. A DSL is a programming-model feature, but it is also a possible target for schedule synthesis. A topology-aware schedule is an algorithmic result, but it is also useless unless the runtime can execute it with the required synchronization and memory semantics. The contribution of the report is therefore to keep these relationships visible.

The technical content is grounded in the original papers. From NCCLX, the report includes CTran, zero-copy transfer, DQPLB, AllToAllvDynamic, scalable initialization, and Fault Analyzer. From MSCCL++, it includes the Primitive API, PortChannel, MemoryChannel, SwitchChannel, DSL Executor, Collective API, and default AllReduce algorithms. From SyCCL, it includes topology symmetry, collective symmetry, sketches, sketch combinations, MILP subschedules, ASTRA-sim-based simulation, two-step synthesis, and synthesis-time results.

The first contribution is a structured reading of the three papers under one common vocabulary. The papers use different terminology because they were written for different purposes. NCCLX speaks in terms of production communication semantics, CTran, transport backends, and tooling. MSCCL++ speaks in terms of API layers, channels, primitives, and portability. SyCCL speaks in terms of sketches, sub-demands, MILP synthesis, and symmetry. This report aligns those terms under programming, workload/evaluation, and runtime layers.

The second contribution is to preserve the reference thesis structure while changing the topic. Chapter 3 is the programming-model chapter, so MSCCL++ is placed there. Chapter 4 is the workload-and-evaluation chapter, so the evaluation methodology and workload characteristics of all three papers are placed there. Chapter 5 is the runtime-and-scheduling chapter, so NCCLX runtime mechanisms and SyCCL synthesis mechanisms are placed there. This keeps the form of the reference thesis while replacing the content with the selected communication papers.

The third contribution is a cross-layer synthesis. Rather than concluding that one of NCCLX, MSCCL++, and SyCCL is the best system, the report argues that a future communication

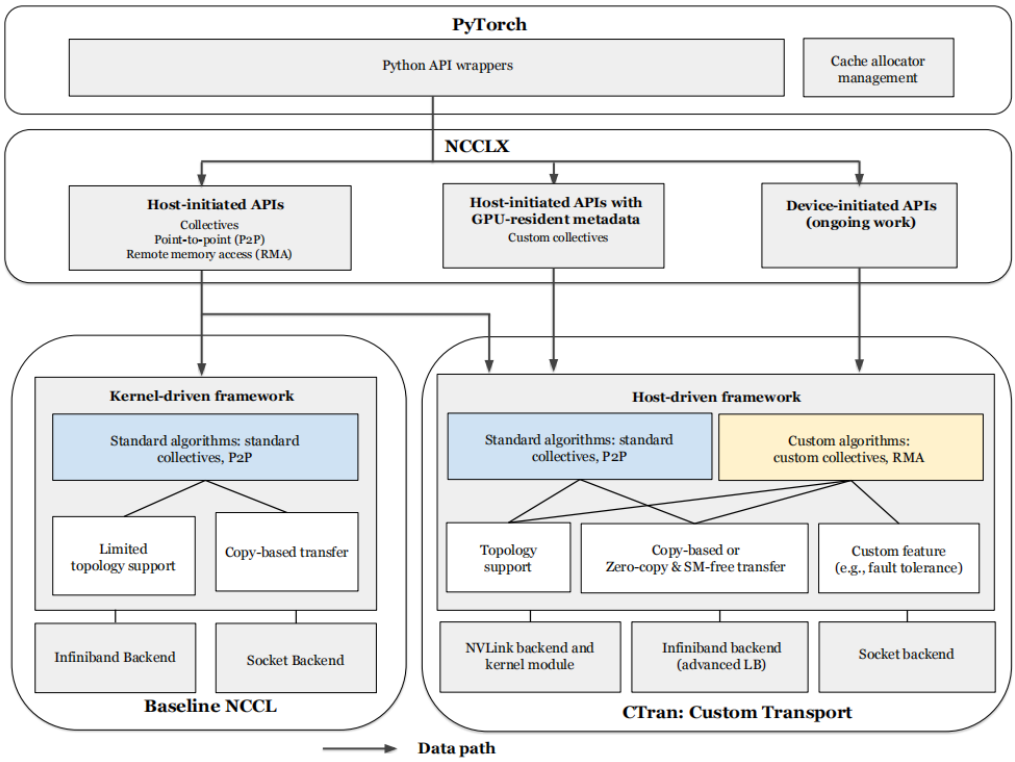


Figure 1.2: NCCLX communication stack overview.NCCLX provides three categories of APIs: Host-initiated APIs, Host-initiated APIs with GPU-resident metadata, and Device-initiated APIs

stack could combine them. A framework could express communication with MSCCL++-like APIs, generate schedules with SyCCL-like synthesis, and deploy them through an NCCLX-like runtime. This is the main conceptual thesis of the paper.

1.5 Methodological Posture

This report is not intended to rank the three systems by a single performance number. Such a ranking would be misleading because the systems solve different problems. NCCLX is evaluated as a production framework for a specific large-scale model family and deployment environment. MSCCL++ is evaluated as a library abstraction that can express and implement optimized algorithms across hardware platforms. SyCCL is evaluated as a synthesizer that improves schedules and reduces search time. A fair comparison must therefore compare problem framing, system boundary, abstraction choice, and engineering implications.

The method I use is close to a systems reading of papers. For each work, I ask five questions. What workload pressure motivates the design? What abstraction does the system expose? What mechanisms implement the abstraction? What evidence supports the design? What limitations or open interfaces remain? This reading style is useful because systems papers often hide their most important contribution not in a single algorithm, but in the choice of boundary between components.

I also distinguish between direct paper content and synthesis. When I describe MSCCL++ primitives such as put, signal, wait, and flush, or NCCLX mechanisms such as DQPLB and AllToAllvDynamic, I am summarizing mechanisms explicitly described in the papers. When I argue that these systems can be combined into one future stack, that is my synthesis. This distinction matters because a course report should not simply reproduce the papers; it should show how the papers relate to each other.

The report intentionally emphasizes system boundaries. For example, MSCCL++ is not only interesting because it reports speedups; it is interesting because it changes the boundary between application developers and communication-library developers. SyCCL is not only interesting because it finds faster schedules; it is interesting because it changes the boundary between generic solvers and domain-specific schedule exploration. NCCLX is not only interesting because it improves Llama4 communication; it is interesting because it expands the boundary of a communication library to include initialization, monitoring, resource management, and fault diagnosis.

This methodological posture also affects how I treat related work. Older systems such as MPI collectives and OpenSHMEM-like one-sided communication provide the conceptual background for collectives, point-to-point transfer, remote memory access, and synchronization [11]. Distributed training systems such as Horovod, Megatron-LM, and ZeRO show how model structure and optimizer structure create communication requirements [8, 5, 7]. MoE systems such as GShard and Switch Transformer explain why dynamic all-to-all communication has become important [4, 2]. I use these references not to replace NCCLX, MSCCL++, and SyCCL, but to show why their mechanisms respond to broader trends.

1.6 Thesis Outline

Chapter 2 introduces the background and related work in collective communication systems. Chapter 3 presents the programming-model layer through MSCCL++ and related abstractions. Chapter 4 defines the workload and evaluation problem through NCCLX, MSCCL++, and SyCCL measurements. Chapter 5 discusses runtime and scheduling, focusing on NCCLX runtime mechanisms and SyCCL schedule synthesis. Chapter 6 synthesizes cross-layer lessons and open challenges. Chapter 7 concludes with future work.

The chapter order exactly follows the reference structure. Chapter 1 motivates the problem, states three research questions, defines the stack, lists contributions, explains methodology, and outlines the report. Chapter 2 provides background and related work. Chapter 3 corresponds to the programming-model contribution. Chapter 4 corresponds to workload and evaluation. Chapter 5 corresponds to runtime and scheduling. Chapter 6 connects the layers and identifies open challenges. Chapter 7 summarizes contributions and future work. The section titles are adapted to the communication topic, but the organization and section counts are intentionally matched to the reference thesis.

The outline also reflects a deliberate choice not to organize the paper as NCCLX first, MSCCL++ second, and SyCCL third. A paper-by-paper organization would be easier to read as a literature survey, but it would hide the layered relationship among the works. In a real system, the programming model, workload definition, and runtime implementation do not execute as independent papers. They interact during every training and inference job. The application expresses a collective or custom communication pattern. The runtime knows topology, registered memory, and transport state. The schedule or algorithm determines which links and buffers are used. The monitoring layer records whether the operation made progress. A layered thesis structure therefore better matches the actual execution structure of a communication stack.

This organization also makes the comparison fairer. NCCLX is a production system whose contributions include initialization, CTran, fault analysis, and inference-specific collectives. MSCCL++ is a programming abstraction whose contributions include primitive operations, channel implementations, and DSL execution. SyCCL is a schedule synthesizer whose contributions include sketch search, symmetry exploitation, MILP sub-scheduling, and simulation. If all three were compared only by collective bandwidth, NCCLX's operational contributions and SyCCL's synthesis-time contributions would be undervalued. If they were compared only by API elegance, NCCLX's transport engineering and SyCCL's topology modeling would be undervalued. The chapter structure avoids forcing one metric onto three different system boundaries.

Another reason for the structure is that communication systems are increasingly becoming infrastructure shared across model families. A single communication stack may support dense training, sparse MoE training, serving prefill, serving decode, checkpointing, and debugging.

Each phase stresses a different part of the stack. A dense training run may care most about bandwidth and overlap. A sparse inference service may care most about small-message latency and dynamic metadata. A failed large-scale job may care most about diagnosis and restart time. A future library that serves all of these phases must be designed as a layered system. The rest of the report uses the three selected papers to show what such a layered system could look like.

Finally, the outline preserves the spirit of the reference thesis by treating the main topic as a systems stack. The reference document studies agentic systems through motivation, background, programming model, workload, runtime, synthesis, and conclusion. This report studies collective communication through the same lenses. The subject is different, but the structural logic is similar: modern systems problems cannot be solved only by an isolated algorithm. They require abstractions, workloads, runtimes, evaluations, and cross-layer interfaces. This is why the report repeatedly connects low-level mechanisms such as queue pairs and memory registration to high-level concerns such as model parallelism, inference latency, and cluster operability.

Chapter 2

Background and Related Work

2.1 LLMs and Collective Communication

Large language models rely on distributed computation because model parameters, activations, optimizer state, and serving throughput requirements exceed the capacity of a single GPU. Collective communication is the mechanism that allows many GPUs to behave as one system. AllReduce aggregates data across ranks, AllGather collects distributed tensors, ReduceScatter distributes reduced results, Broadcast distributes one source to many destinations, and AllToAll supports patterns such as MoE token exchange.

In data-parallel training, AllReduce historically occupied the center of the communication discussion because every worker computes gradients and the system must combine them before updating parameters. Horovod made this pattern easy to use by integrating ring-allreduce-style communication with popular deep learning frameworks [8]. In modern LLM training, however, data parallelism is only one part of the picture. Tensor parallelism places collectives inside the forward and backward computation of each transformer block. Sequence parallelism and sharded optimizer strategies move tensors at different points of the step. Pipeline parallelism introduces point-to-point activation traffic. Expert parallelism introduces token exchange. The result is that a model step contains several communication phases with different sizes, synchronization requirements, and overlap opportunities.

The importance of communication increases with model parallelism. Tensor parallelism creates communication inside transformer layers. Pipeline parallelism creates point-to-point traffic between stages. Expert parallelism creates dynamic all-to-all communication. Sharded data parallelism creates large collectives in outer parallel domains. In inference, decoding latency makes even small CPU overheads visible. These workload facts explain why NCCLX, MSCCL++, and SyCCL all go beyond fixed NCCL-style collectives.

Megatron-LM is an important background point because it shows that communication is not only a final gradient synchronization step [5]. Tensor-parallel transformer layers require synchronization around attention and feed-forward operations, and these collectives can sit directly on the critical path of every layer. ZeRO-style partitioning changes the memory/communication trade-off by reducing replicated state, but it also requires collectives or gather/scatter operations to materialize data when needed [7]. MoE models add a different pattern: tokens are routed to experts, so communication volume depends on the router output rather than only on a static tensor shape [4, 2]. This is exactly the kind of dynamic metadata problem that motivates NCCLX's GPU-resident collective support.

The communication patterns differ not only in the operation name but also in the performance objective. AllReduce for large gradients is often bandwidth-dominated. AllGather for small parameters or metadata may be latency-dominated. Pipeline send/receive can sit directly before or after GEMM kernels, so using GPU SMs for communication may reduce compute throughput. MoE AllToAll can involve only a few megabytes per operation, but the CPU overhead of preparing many network requests can dominate. These differences explain why a single fixed algorithm cannot serve as a universal solution.

The physical cluster also matters. Intra-node communication may use NVLink, xGMI, or PCIe. Inter-node communication may use InfiniBand, RoCE, or Ethernet. The ratio between intra-node and inter-node bandwidth determines whether a ring, tree, hierarchical schedule, or synthesized schedule performs best. SyCCL explicitly uses topology dimensions and groups to exploit repeated structures; NCCLX uses CTran backends and DQPLB to adapt to scale-out network conditions; MSCCL++ exposes channel types so that different hardware transfer modes can be used without rewriting the whole stack.

Classical collective algorithms already recognized that topology matters, but GPU clusters make the topology more heterogeneous. A single node may contain a dense NVLink fabric, while communication across nodes may pass through NICs, PCIe switches, top-of-rack switches, and spine switches. The latency and bandwidth ratios between these paths can be large. If a collective schedule treats all links as equal, it may overload the slow path or leave the fast path idle. If it over-specializes to one topology, it may fail when deployed on a different cluster. This tension motivates SyCCL's symmetry-based synthesis: repeated structure is exploited, but the topology is still represented explicitly [1].

2.2 Programming Models and Frameworks

Traditional frameworks expose high-level collective APIs. This is easy for application developers but can hide the hardware behaviors needed for optimization. MSCCL++ directly addresses this limitation by proposing three hierarchical user interfaces: Primitive API, DSL API, and Collective API [9]. The Primitive API exposes communication operations such as put, signal, wait, and flush inside GPU kernels. The DSL API allows custom algorithms to be described at a higher level. The Collective API preserves NCCL-style compatibility.

The high-level API tradition is useful because it makes distributed programs portable. MPI collectives established the idea that applications should express communication intent through operations such as Broadcast, Reduce, AllReduce, and AllGather rather than manually coding every send and receive [11]. NCCL brought the same style to GPU-centric training systems with implementations tuned for NVIDIA interconnects [6]. The limitation is that the collective call boundary can become too coarse. It hides whether a message is staged, whether synchronization occurs on the CPU or GPU, whether the operation consumes SM resources, and whether routing metadata can remain on the GPU. MSCCL++ responds by adding lower layers without discarding the high-level surface.

NCCLX also broadens the programming model by supporting host-initiated APIs, host-initiated APIs with GPU-resident metadata, and device-initiated APIs [10]. These modes correspond to different points in the expressiveness-performance trade-off. A standard collective can remain host-driven, while MoE inference can use GPU-resident metadata produced by a router kernel. The related NCCL Device API work further motivates this direction by enabling GPU kernels to initiate communication more directly [3].

The programming-model problem is a balance between hiding and exposing hardware. NCCL-style APIs hide hardware details, which is ideal for common training loops. But hiding too much can prevent applications from using new hardware features or custom schedules. MSCCL++ addresses this by making the primitive interface shallow: it abstracts low-level communication concepts without pretending all interconnects are identical. Its PortChannel, MemoryChannel, and SwitchChannel are examples of abstractions that preserve hardware differences while giving developers a common vocabulary.

There is also a compatibility problem. A new programming model is difficult to adopt if it requires all model code to be rewritten. MSCCL++ therefore preserves a Collective API compatible with NCCL. NCCLX similarly operates below PyTorch rather than asking model developers to manage every transport detail. This indicates that future communication programming models must be incremental: they must support existing applications while enabling experts to specialize where necessary.

Compatibility is not only an ergonomic issue; it is an experimental-control issue. If a new communication library requires changing the model implementation, then performance differences

may come from model-code changes rather than the communication layer. NCCL-compatible and PyTorch-compatible surfaces make it possible to compare communication mechanisms while keeping model semantics stable. This is one reason the selected papers emphasize compatibility even when their novel mechanisms sit below it. They need a path into existing training and inference stacks.

2.3 Benchmarks and Evaluation

Evaluating collective communication is multi-dimensional. A microbenchmark may measure latency or algorithmic bandwidth for one collective and message size, but production systems also care about training step time, inference decoding latency, synthesis time, initialization time, and failure recovery. MSCCL++ separates small-message latency from large-message bandwidth and evaluates A100, H100, and MI300x environments. SyCCL evaluates schedule performance for AllGather, AllToAll, and ReduceScatter across sizes from 1KB to 4GB and across topologies from single-server to hundreds of GPUs [1]. NCCLX reports training and inference improvements, initialization speedups, and tooling benefits in production-scale settings [10].

This diversity of metrics is why the report does not use one performance number as the conclusion. The right metric depends on the layer. A programming model is judged by expressiveness, overhead, and portability. A workload evaluation is judged by whether it reflects real training and inference bottlenecks. A runtime is judged by sustained performance, scalability, and failure handling.

The selected papers also show that evaluation must include engineering cost. MSCCL++ reports that adding AMD MI300x support took seven weeks for one developer, and that adding SwitchChannel support for NVLink 4.0 multimem took eight weeks for two developers. These numbers matter because a communication library is maintained over hardware generations. A system that is fast but requires a hard fork for each platform may not be sustainable.

SyCCL's evaluation introduces another dimension: synthesis time. A schedule that is excellent but takes many hours to generate may be impractical for production use. SyCCL therefore compares not only schedule performance but also generation time, reporting large reductions compared with TECCL. NCCLX adds yet another dimension: initialization time. At 96K or 100K+ GPU scale, startup and restart time become part of training efficiency. These evaluation choices broaden the meaning of performance.

2.4 Runtime and Scheduling

Runtime and scheduling are where logical communication semantics become physical data movement. NCCLX uses CTran as a custom transport framework with NVLink, InfiniBand/RoCE, and socket backends. It supports zero-copy and SM-free communication when possible, manages tensor registration, and uses DQPLB to balance traffic across queue pairs. SyCCL addresses scheduling from a different direction: it synthesizes collective schedules by decomposing demand into sketches, solving sub-schedules, simulating candidates, and selecting the best schedule.

The runtime problem is difficult because no single mechanism dominates all cases. Large messages need bandwidth and congestion control. Small messages need low latency and low CPU overhead. MoE inference needs dynamic metadata. Large training jobs need scalable initialization and fault localization. The three selected papers collectively show that runtime support must be adaptive rather than fixed.

For NCCLX, runtime begins before the first model step. Communicators must be created, metadata must be exchanged, buffers may need registration, and network resources must be allocated. The paper shows that baseline initialization can become non-linear and serialized at large scale. Its scalable initialization work therefore belongs in the runtime layer. Fault Analyzer also belongs here because runtime failure behavior determines whether long training runs can recover or be diagnosed.

For SyCCL, runtime has an offline or preparation component. Sketch exploration and schedule synthesis may run before execution, but their output determines runtime data movement. The schedule simulator is used to select among candidates without executing all candidates on hardware. For MSCCL++, runtime appears through the DSL Executor and channel implementations, which execute the communication program produced by higher layers. Together, these systems show that runtime includes both preparation and execution.

2.5 Existing Surveys

Existing discussions of GPU communication often focus on NCCL, MPI, NVSHMEM, or hardware interconnects. These are essential foundations, but the three selected papers show a newer layer of system design. NCCLX is not merely a new collective; it is a production communication framework. MSCCL++ is not merely a new AllReduce kernel; it is a layered programming interface. SyCCL is not merely a faster schedule for one topology; it is a synthesis method that exploits symmetry.

The evolution from MPI-style collectives to GPU-resident and device-initiated communication can be understood as a shift in where control lives. Traditional systems often let the CPU orchestrate communication and use the accelerator mainly for computation. In GPU-centric LLM systems, the CPU can become the bottleneck for small and frequent operations. GPU-initiated networking and NCCL Device API work therefore move more control toward GPU kernels [3]. MSCCL++ moves programming control into GPU kernels through primitives, and NCCLX moves some inference metadata into GPU-resident collective paths. These efforts are connected by the same goal: reduce unnecessary host involvement when the GPU already has the information needed to communicate.

This report differs from a broad survey by using three representative systems to build a coherent stack. The goal is not to cover every communication library, but to explain how programming, workload evaluation, and runtime scheduling interact in modern AI communication systems.

The papers also relate to older high-performance computing ideas. MPI and OpenSHMEM have long exposed collectives, point-to-point operations, and one-sided communication. However, GPU-centric AI workloads change the trade-offs. Kernel launch overhead, GPU memory registration, CUDA graph compatibility, SM usage, and NVLink/IB/RoCE topology all become central. The systems discussed here can be seen as adapting HPC communication ideas to the execution model and operational scale of modern LLM training and serving.

Another related direction is compiler-runtime co-design. Model compilers know tensor shapes, parallelism groups, and operation order. Communication libraries know transports and synchronization. Schedule synthesizers know topology and collective demand. A future system could combine these sources of information. This report does not implement such a compiler, but the cross-layer synthesis in Chapter 6 points toward it.

2.6 Summary

This chapter positioned the selected papers in the context of LLM communication. The key background lesson is that collective communication now spans multiple concerns: programming interfaces, workload-specific evaluation, topology-aware scheduling, and production runtime support. The rest of the report follows the same layered structure as the reference thesis: programming model, workload and evaluation, runtime, cross-layer synthesis, and conclusion.

It is also useful to state what is not sufficient. Merely replacing one collective algorithm with another does not solve the whole problem. A ring may be good for bandwidth in one topology but bad for small-message latency in another. A tree may reduce latency but underuse bandwidth for large data. A custom kernel may improve one model but be hard to port to a new GPU. A synthesized schedule may be fast but difficult to deploy if the runtime cannot express it. This is why the following chapters keep returning to interfaces: the interface between

application and library, between synthesizer and executor, between transport and topology, and between runtime and operator.

The three papers also show how the definition of "communication library" is expanding. In a narrow definition, a communication library implements collective operations. In the broader definition used here, it also manages memory registration, topology information, control-plane initialization, algorithm selection, schedule synthesis, and failure analysis. This broader definition better matches the actual requirements of LLM training and inference.

A useful way to understand this expansion is to compare the assumptions of earlier deep learning communication with the assumptions of current LLM systems. Earlier systems often assumed that the main recurring operation was gradient AllReduce over a data-parallel group. This operation was large enough to be bandwidth dominated and regular enough to be optimized by a small set of algorithms. Modern LLM systems break both assumptions. Tensor parallelism creates frequent collectives whose sizes are tied to hidden dimension, sequence length, and layer structure. Pipeline parallelism creates send/receive traffic whose timing interacts with microbatch scheduling. Expert parallelism creates dynamic all-to-all exchange whose sizes are not known until routing is computed. Serving systems introduce graph capture and latency constraints. As a result, the communication library is no longer just a collection of optimized collectives; it is part of the execution substrate of the model.

The background also suggests why topology-awareness cannot be treated as an optional optimization. GPU clusters are built from repeated but unequal domains. Inside a node, GPUs may communicate through high-bandwidth NVLink or xGMI. Across nodes, traffic may pass through NICs, PCIe, and a multi-level network fabric. Across racks, bandwidth and congestion behavior can change again. A topology-agnostic ring can be robust and simple, but it may impose a traffic pattern that conflicts with the physical bandwidth hierarchy. A topology-aware algorithm can be faster, but hand-designing one for every topology is not scalable. SyCCL's symmetry-based approach is important because it looks for reusable structure inside topology-specific optimization [1].

The related-work landscape can also be divided by control location. NCCL-style collectives expose a host API and hide most details behind the library. GPU-initiated networking shifts more initiation responsibility toward GPU kernels [3]. MSCCL++ exposes GPU-side primitives but still allows CPU involvement where hardware requires it [9]. NCCLX supports host-initiated, host-initiated with GPU-resident metadata, and device-initiated paths [10]. These are not mutually exclusive choices; they are points in a design space. The right control location depends on whether the operation is large or small, static or dynamic, graph-captured or not, and latency-sensitive or throughput-sensitive.

Another important background theme is memory visibility. GPU communication involves more than sending bytes. The receiver must know when bytes are valid. The sender must know when a buffer can be reused. The runtime must know whether data is visible across devices and whether synchronization is required. One-sided communication makes this especially subtle because the receiver may not actively participate in every transfer. MSCCL++ primitives such as signal, wait, and flush make these ordering points explicit. NCCLX's tensor registration management makes buffer validity and memory-region registration explicit. SyCCL's schedule constraints make dependency ordering explicit. These three systems therefore address correctness as well as performance, even when their headline results emphasize speed.

The background chapter also clarifies the difference between standardization and stagnation. Standard collective APIs are valuable because they make applications portable and easy to write. However, if the standard interface is the only interface, new workload patterns force developers into ad hoc private communication code. The challenge is to standardize the stable part of the interface while still leaving an extension path. MSCCL++'s hierarchy is one possible answer: keep a collective API for ordinary users, add a DSL for algorithm designers, and expose primitives for backend and kernel experts. NCCLX takes a different but related approach by preserving integration with PyTorch and NCCL semantics while extending the runtime below them.

Finally, existing surveys often separate algorithmic communication from production deploy-

ment. The selected papers show that this separation is increasingly artificial. A schedule that is fast in a simulator must still be deployed through a runtime. A primitive API that is expressive must still initialize communicators and register buffers. A production runtime that is robust must still choose good algorithms. The background chapter therefore treats related work as a set of partial answers rather than a linear history. MPI and NCCL provide standard collective foundations; Horovod, Megatron-LM, ZeRO, GShard, and Switch Transformer define workload pressure; NCCLX, MSCCL++, and SyCCL show how the communication layer is responding.

One additional background distinction is between logical collectives and physical traffic. A logical AllReduce has a simple mathematical meaning: all ranks contribute values and all ranks receive the reduced result. Physically, however, the operation may be implemented as a ring, a tree, a reduce-scatter plus all-gather, a hierarchical intra-node/inter-node schedule, or a synthesized multi-stage schedule. These implementations differ in the number of steps, bytes sent over each link, buffering requirements, reduction placement, and synchronization points. This distinction matters because many discussions of communication use the operation name as if it uniquely determines performance. The selected papers show the opposite: the operation name defines semantics, while performance depends on how the runtime maps those semantics onto topology and hardware.

Another background issue is that communication is increasingly intertwined with memory management. ZeRO-style sharding changes where parameters, gradients, and optimizer states reside [7]. Tensor parallelism and pipeline parallelism change when activations must be exchanged [5]. MoE routing changes which token buffers must move at runtime [4, 2]. These placement decisions determine whether a communication operation can use a contiguous buffer, whether it needs packing, whether the buffer is already registered, and whether the result can be consumed without additional copies. NCCLX's tensor registration work is therefore not a narrow implementation trick; it is a response to the fact that the communication layer now depends on framework memory behavior [10].

It is also useful to separate algorithmic latency from orchestration latency. Algorithmic latency is the time implied by the schedule: number of hops, link latencies, synchronization steps, and transmission time. Orchestration latency is the time needed to prepare and launch the operation: CPU callbacks, metadata creation, stream synchronization, graph breaks, proxy-thread work, and kernel launches. In large-message training, algorithmic bandwidth may dominate. In small-message inference, orchestration latency may dominate. MSCCL++ attacks orchestration overhead by moving more communication expression into GPU-side kernels and reusable primitives [9]. NCCLX attacks it by supporting GPU-resident metadata and device-initiated paths for inference-sensitive operations [10]. SyCCL mainly attacks algorithmic schedule quality, but its schedules still need an execution layer with low orchestration overhead.

The related-work story also explains why automatic schedule synthesis is difficult. A collective schedule must be legal, fast, and implementable. Legal means that dependencies and reductions are correct. Fast means that the schedule uses topology well under a cost model. Implementable means that the runtime can express the required transfers, synchronization, and buffer use. TECCL-style approaches and SyCCL-style approaches address the search problem, but the final schedule still has to run inside a communication library. This is why Chapter 6 emphasizes an intermediate representation: without a common representation, schedule synthesis remains separate from production execution.

Finally, the background suggests a practical warning. It is tempting to treat new communication systems as replacements for NCCL. The more accurate view is that they extend or surround NCCL-like functionality. NCCL remains essential because it provides a reliable, optimized baseline for standard collectives [6]. MSCCL++ preserves NCCL-compatible APIs because replacing the whole application interface would be unrealistic [9]. NCCLX builds production mechanisms around NCCL and CTran rather than discarding existing framework integration [10]. SyCCL compares against NCCL because NCCL represents the standard high-quality baseline [1]. The future stack is therefore evolutionary: it keeps stable interfaces while adding programmability, synthesis, and operability below or beside them.

Chapter 3

Programming Model: Programmable Collective Communication

3.1 Motivation

The programming-model layer asks how developers express communication. A single high-level collective API is convenient, but it is too rigid for cutting-edge AI workloads. MSCCL++ starts from the observation that production applications often write custom communication code because general-purpose libraries need time to adapt to new hardware and workload patterns [9]. This produces high performance in the short term but also creates duplicated, non-portable engineering work.

The central motivation is therefore separation of concerns. Low-level primitives should expose enough hardware behavior for experts to optimize. Higher-level interfaces should preserve portability and usability. This is analogous to the reference thesis's programming-model layer: the goal is to identify the abstraction that makes a complex system programmable without hiding the important control surface.

Separation of concerns also reduces duplication. Without a reusable primitive layer, each application that needs a custom collective may implement its own transport logic, synchronization protocol, memory-registration handling, and algorithm selection. This produces short-term performance but long-term fragmentation. With a primitive layer, custom algorithms can share the same backend implementation. With a DSL layer, algorithm designers can express data movement without writing every low-level instruction. With a collective layer, ordinary users can benefit from optimized algorithms without knowing how they are implemented. This is the same systems principle that underlies compilers and databases: expose high-level intent, but keep lower-level hooks where expert control is necessary.

MSCCL++ argues that existing abstractions often hide too much. A high-level collective call is easy to use, but it prevents the programmer from controlling when synchronization happens, whether data transfer is one-sided, whether communication can be fused into computation, or whether a new hardware feature can be used. At the opposite extreme, writing directly against vendor-specific low-level APIs produces non-portable code. The programming-model contribution of MSCCL++ is to put a thin, reusable layer between these extremes.

This motivation is not purely academic. The paper notes that real AI applications often develop quick custom communication stacks for their own workloads and hardware. TensorRT-LLM, for example, contains custom AllReduce methods that can outperform NCCL in certain LLM scenarios. The lesson is not that standard libraries are unnecessary; it is that cutting-edge applications need a path to specialize before general libraries fully incorporate every optimization. MSCCL++ tries to make that specialization reusable.

3.2 The Communicable-Primitive Abstraction

The central abstraction in MSCCL++ is the primitive communication interface. It exposes simple operations such as put, signal, wait, and flush. A put operation moves data to a remote location. A signal notifies a peer or updates a synchronization value. A wait operation blocks until a condition is satisfied. A flush controls completion and visibility. These operations are close to GPU hardware behavior and support zero-copy, one-sided, and asynchronous communication [9].

MSCCL++ implements these primitives through channels. A PortChannel handles transfers over ports connected to GPU memory, such as RDMA. A GPU writes a request into a FIFO queue shared with a CPU thread, and the CPU thread processes requests such as RDMA puts or atomic signal operations. A MemoryChannel supports direct GPU memory access when the interconnect allows load/store-style communication. A SwitchChannel supports group-level hardware features such as NVLink 4.0 multimem instructions for aggregation and multicast. This channel design is shallow enough to remain close to hardware but structured enough to support portability.

The PortChannel workflow illustrates the design. When a GPU calls put, the first participating GPU thread pushes a request into a queue. The CPU thread polls the queue, reads the request, and starts an asynchronous transfer such as ibv_post_send. A signal operation similarly becomes a request processed by the CPU, often implemented as an atomic increment of a semaphore on the receiving GPU. The receiving GPU's wait primitive does not need to wake the receiving CPU; it waits for the semaphore value. This workflow uses the CPU where the hardware still requires CPU initiation, but keeps the programming interface inside the GPU kernel.

MemoryChannel and SwitchChannel show the opposite case: when hardware can support more direct operations, the abstraction can expose them without changing the upper layers. MemoryChannel is appropriate when GPUs can directly load from or store to peer memory. SwitchChannel is appropriate when the interconnect switch itself supports multicast or aggregation. This is why the channel abstraction is not simply an implementation detail; it is the mechanism by which MSCCL++ remains portable across interconnect modes.

The channel abstraction also gives a clean way to discuss progress and completion. In CPU-mediated RDMA, progress may depend on a polling thread or a network interface processing posted work requests. In peer-memory access, progress may be tied more directly to GPU instructions and memory ordering. In switch-assisted communication, the interconnect can perform operations that previously required multiple point-to-point transfers. A single abstract collective call hides these distinctions. MSCCL++ does not expose every hardware register, but it exposes enough channel structure for algorithms to be matched to transfer mechanisms. This is why the primitive layer is not just a lower-level API; it is also a vocabulary for reasoning about hardware capabilities.

Figure 3.1 emphasizes the main programming-model distinction: the user does not have to choose between a single rigid collective API and completely hardware-specific code. Instead, the same stack exposes a compatibility layer, a DSL layer, and a primitive layer. This is why MSCCL++ is used as the programming-model chapter rather than only as a performance case study.

3.3 Communication Programming

On top of the Primitive API, MSCCL++ provides a Python-based DSL API. The DSL describes a communication algorithm at a high level and converts it into instructions executed by a GPU DSL Executor. The executor reads and runs these instructions back-to-back. The paper notes that this extends MSCCLang and lifts restrictions such as allowing a single GPU thread block to access multiple GPUs at the same time [9].

MSCCL++ also provides a Collective API that reimplements the familiar NCCL API. This is the lowest-effort path for users who want to replace NCCL/RCCL without changing appli-

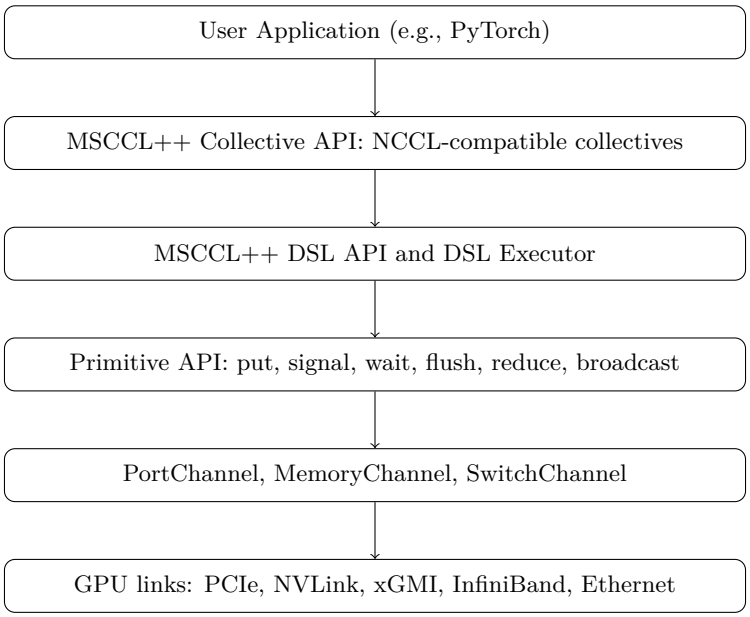


Figure 3.1: MSCCL++ stack redrawn from the description of Figure 3 in the MSCCL++ paper [9].

cation code. The three API levels provide a continuum. The Primitive API gives maximum control, the DSL API gives algorithmic flexibility with lower effort, and the Collective API gives compatibility. This directly matches the systems design principle that one stack should support multiple expertise levels.

Initialization is part of the programming model as well. MSCCL++ programs run as multiple processes, usually one process per GPU. The host-side bootstrap interface exchanges metadata between processes through methods such as send, receive, allGather, and barrier. The default bootstrap uses POSIX sockets, but the design allows replacement with MPI or torch.distributed. After metadata exchange, a communicator registers buffers and constructs channels based on physical GPU links. This initialization design is important because the primitive API depends on channels and buffers being established before kernels execute.

This bootstrap detail is easy to overlook, but it is essential for programmable communication. A primitive such as put is simple only after the runtime has already established which peer memory region is valid, which transport should be used, which synchronization location is associated with the transfer, and which GPU or CPU component will make progress. In other words, the primitive API is small because the communicator setup phase has already installed the necessary context. This mirrors NCCLX's emphasis on initialization: at scale, the setup phase is part of the system, not a negligible prelude [10].

The DSL Executor is another key programming mechanism. The DSL is Python-based and converts a high-level algorithm description into instructions. A GPU kernel then reads and executes those instructions. This is a compromise between hand-written kernels and fixed library calls. It lets users describe custom collective schedules while still relying on the primitive layer for execution. The cost is possible executor overhead, but the benefit is faster algorithm development and better portability.

3.4 Applications

The paper illustrates the programming model through collective kernels. One-phase all-pairs AllReduce sends each GPU's local data to all other GPUs and performs reduction in one phase. It is suitable for very small messages where synchronization overhead dominates redundant traffic. Two-phase all-pairs splits AllReduce into ReduceScatter and AllGather, reducing redundant

data movement and improving bandwidth efficiency. Two-phase hierarchical algorithms perform local collectives inside nodes and minimize cross-node traffic.

These applications show why low-level programmability matters. For small messages, reducing synchronization may matter more than minimizing bytes. For multi-node messages, reducing cross-node traffic may dominate. MSCCL++ lets developers select channels, protocols, and algorithms according to message size and hardware. The same stack can express default collectives, DSL-defined algorithms, and hand-written primitive kernels.

The AllReduce examples are especially useful. One-phase all-pairs (1PA) broadcasts all local data to all other GPUs and reduces in one phase. This sends redundant data, but it avoids extra synchronization, making it suitable for very small messages. Two-phase all-pairs (2PA) splits the operation into ReduceScatter and AllGather. This reduces redundant traffic and reduction work, making it better for larger messages. Two-phase hierarchical (2PH) performs local collectives inside each node and minimizes cross-node exchange, which is beneficial when inter-node bandwidth is more constrained than intra-node bandwidth.

These examples illustrate an important point about algorithm selection: the asymptotically cleaner algorithm is not always faster at the sizes that matter. A redundant all-pairs algorithm may look wasteful in bytes, but if it removes a synchronization round and the message is only a few kilobytes, it can win. A hierarchical algorithm may reduce inter-node bytes, but if the intra-node phase is poorly implemented or if the hierarchy mismatches the hardware, it can lose. This is why MSCCL++ evaluates across message sizes rather than reporting a single headline number. The programming model must support multiple algorithms because the workload moves across latency-dominated and bandwidth-dominated regimes.

MSCCL++ can implement these variants with different channels and protocols. For example, it can use MemoryChannel with a low-latency protocol for small intra-node collectives, or PortChannel for RDMA-based inter-node transfers. It can also use rotating buffers or allow one thread group to read data from multiple GPUs, reducing synchronization compared with implementations that read peers one-by-one. These are precisely the kinds of optimizations that are hard to express with only a monolithic collective API.

3.5 Discussion

MSCCL++ also provides concrete evidence for portability. Adding AMD MI300x support reportedly took seven weeks for one developer, with less than ten lines of AMD-specific code excluding build files and algorithms [9]. Adding SwitchChannel for NVLink 4.0 multimem took eight weeks for two developers and enabled a new AllReduce algorithm that outperformed NCCL/MSCCL by more than 2.2 times on average for 1KB–1GB message sizes. These numbers are important because they measure engineering effort as well as runtime performance.

The main open question is correctness. Low-level communication primitives involve memory ordering, synchronization, and cross-device visibility. A future MSCCL++-style stack may need validators, race detectors, or schedule checkers. Another open question is synthesis: a SyCCL-like scheduler could generate MSCCL++ DSL programs, combining automatic optimization with programmable execution.

The evaluation also shows the performance side of the design. MSCCL++ reports collective communication speedups up to 5.4 times and AI inference speedups up to 15 percent. It compares against NCCL, RCCL, and MSCCL, and it tunes baselines across message sizes and environments. The paper separates small-message latency from large-message bandwidth because the relevant bottleneck changes with size. Small messages resemble inference decode, where latency dominates. Large messages resemble training or inference prefill, where bandwidth dominates.

The portability results are just as important as the raw speedups. A library that only works well on one hardware generation cannot serve as a long-term systems layer. MSCCL++'s shallow primitive abstraction makes it easier to add AMD support and to add new NVIDIA multimem support. This suggests that the programming model can reduce maintenance cost, not only runtime cost.

There is also an important distinction between portability of user programs and portability of optimized algorithms. The Collective API gives user-program portability because applications can keep NCCL-like calls. The DSL API gives algorithm portability because the same algorithm description can be executed through primitives on different hardware. The Primitive API gives implementation portability because backend developers can implement a small set of operations for a new platform. These three forms of portability reinforce each other but are not identical.

The cost of this design is that performance tuning becomes multi-level. A slowdown might come from the algorithm expressed in the DSL, the executor overhead, the primitive implementation, the channel choice, or the hardware backend. This is why good profiling and debugging support would be necessary in a mature MSCCL++ ecosystem. Nevertheless, the layering makes the problem diagnosable: each layer has a clear role, unlike ad hoc custom code where algorithm, transport, and synchronization may be tangled together.

This multi-level tuning problem suggests a natural connection to schedule synthesis. A SyCCL-like synthesizer could search over high-level schedule structures, while an MSCCL++-like runtime could lower selected schedules into primitives. The synthesizer should not need to know every backend detail, but it should know enough cost information to avoid schedules that are impossible or inefficient on a given channel. Conversely, the primitive layer should expose enough performance metadata for the synthesizer to choose between channel types. This would make programming and synthesis part of one optimization loop rather than separate manual processes.

3.6 Summary

This chapter presented MSCCL++ as the programming-model layer of the collective communication stack. Its contribution is not only speedup, but a layered interface that lets users move between NCCL-compatible collectives, DSL-defined algorithms, and low-level primitives. This layer supplies the expression mechanism that later workload and runtime layers need.

The programming-model lesson can be summarized as controlled exposure. A communication library should not expose every hardware detail to every user, because that would make application development too difficult. But it should expose enough structure that experts can implement new algorithms without forking the entire stack. MSCCL++ does this by making primitives reusable across higher-level interfaces. The same put, signal, wait, and flush concepts can support hand-written kernels, DSL-generated kernels, and collective-library kernels.

The channel design is also a form of controlled exposure. PortChannel exposes the fact that some transfers still require CPU initiation. MemoryChannel exposes the fact that some GPUs can access peer memory directly. SwitchChannel exposes the fact that some interconnects can perform switch-assisted aggregation or multicast. Instead of flattening all of these into one abstract send/receive operation, MSCCL++ gives each transfer mode an explicit place in the programming model.

One practical implication is that communication optimization becomes more modular. A hardware vendor can add or optimize a channel. A communication expert can write a DSL algorithm. An application developer can keep using a collective API. This is more sustainable than the ad hoc pattern where every application carries a private communication implementation. It also makes it easier to test and compare algorithms, because different implementations can share the same primitive substrate.

At the same time, MSCCL++ does not remove the need for expertise. Choosing between 1PA, 2PA, and 2PH algorithms still requires understanding message size, topology, synchronization cost, and bandwidth. The DSL lowers the implementation burden, not the reasoning burden. This is why an automatic synthesizer such as SyCCL is complementary: it can help choose or generate algorithms that a DSL can express.

Finally, the programming-model chapter highlights why backward compatibility matters. The Collective API is not the most novel part of MSCCL++, but it may be the most important path for adoption. Production frameworks cannot rewrite all model code whenever a communication library changes. By preserving NCCL-style calls while enabling optimized implementations

underneath, MSCCL++ follows a common systems strategy: keep the external interface stable while improving the internal execution path.

This programming model also changes who can contribute optimizations. In a traditional closed collective library, only library maintainers can easily add algorithms. In an ad hoc custom stack, each application team adds its own private algorithm. In MSCCL++, a communication expert can write a DSL algorithm, a hardware expert can optimize a channel or primitive, and an application developer can use the collective interface. This separation of roles is one of the strongest arguments for MSCCL++ as a systems contribution.

The chapter therefore answers the first research question: collective communication should be programmed through a hierarchy rather than a single interface. The hierarchy must include a compatible top layer for adoption, a declarative middle layer for algorithm development, and a primitive lower layer for hardware-specific optimization. Without the top layer, the system is hard to adopt. Without the middle layer, custom algorithm development remains too slow. Without the lower layer, the system cannot expose new hardware capabilities.

One additional lesson from the programming-model layer is that communication programs need to describe both movement and synchronization. A data-movement operation alone is not enough. The sender and receiver must agree on when a transfer starts, when it completes, when a reduction result is visible, and when a buffer can be reused. In a CPU-orchestrated library, much of this ordering can be hidden behind blocking or stream-ordered collective calls. In a GPU-resident primitive interface, ordering becomes part of the program. MSCCL++ therefore includes signal, wait, and flush as first-class operations rather than treating them as incidental implementation details [9]. This is a major reason why the primitive interface is more than a wrapper around memcpy or RDMA put.

The hierarchy also lets a communication stack evolve with hardware. Suppose a new GPU interconnect adds a switch-level reduction instruction, a new multicast mechanism, or a lower-latency memory operation. A monolithic collective library must either hide the feature until maintainers implement new collectives, or expose a vendor-specific escape hatch. A primitive/channel hierarchy offers a middle path. Backend developers can implement a new channel or extend an existing one; algorithm developers can use the new capability in DSL programs; application developers can continue calling collectives. MSCCL++'s SwitchChannel support for NVLink 4.0 multimem is an example of this pattern [9].

Another programming-model issue is how much responsibility should be placed on the user. The Primitive API is powerful, but not every user should write primitive-level communication kernels. Incorrect synchronization or memory-ordering assumptions can produce rare and hard-to-debug errors. The DSL API reduces this burden by allowing algorithms to be expressed at a higher level. The Collective API reduces it further by preserving familiar calls. This means the programming model is not simply a vertical stack of performance options; it is also a stack of responsibility. Users can choose the lowest layer they need, and the system can provide safer defaults at higher layers.

This responsibility gradient also supports incremental adoption. A framework can begin by replacing NCCL-compatible calls with MSCCL++ Collective API implementations. Later, performance-critical operations can be rewritten as DSL algorithms. Only the most specialized or latency-critical paths need Primitive API kernels. This adoption path is realistic for production systems, where rewriting every model and every communication call at once would be too risky. It also allows performance engineering to focus on the operations that actually matter in profiling, rather than forcing all communication through custom code.

The programming model therefore has a social dimension as well as a technical one. Different engineers own different layers. Model developers own model code and high-level parallelism choices. Communication experts own algorithm selection and schedule design. Runtime engineers own transport backends, memory registration, and progress mechanisms. Hardware vendors own low-level capabilities and driver behavior. A good programming model gives these groups a shared interface without forcing all of them into the same code layer. This is why MSCCL++ is a systems contribution even when viewed apart from its speedup numbers.

The chapter also suggests several requirements for future tools. A DSL compiler should be

able to check that waits correspond to signals, that buffers are not reused too early, and that reductions are applied correctly. A profiler should attribute time to DSL instructions, primitive operations, channel backends, and transport queues. A simulator or cost model should estimate whether a DSL schedule will be latency-bound or bandwidth-bound before it runs on hardware. These tools would make programmable communication safer and more accessible. Without them, the low-level power of a primitive API could remain limited to a small group of experts.

Finally, MSCCL++ helps clarify the relationship between programming models and automatic synthesis. A synthesizer such as SyCCL needs a target language. If the target is too high-level, the synthesizer can only choose among existing collectives. If the target is too low-level, the synthesizer must reason about backend-specific details that make portability difficult. A DSL plus primitive layer is a plausible middle target. SyCCL could produce a schedule structure, the DSL could represent it, and primitives could execute it through PortChannel, MemoryChannel, or SwitchChannel. This is the clearest cross-layer bridge between Chapter 3 and Chapter 5.

The programming model should also expose enough metadata to support selection. A collective call such as AllReduce normally provides a buffer, count, datatype, reduction operator, communicator, and stream. For advanced selection, the runtime may also need to know whether the operation is in training or inference, whether the message is latency-sensitive, whether graph capture is active, whether the buffer is reused across iterations, and whether communication can consume SMs. Some of this information can be inferred, but explicit metadata would reduce ambiguity. NCCLX's distinction among host-initiated, host-initiated with GPU-resident metadata, and device-initiated APIs suggests that future programming models may need richer call surfaces for advanced cases [10].

A further issue is composability. Modern model code often composes several parallelism strategies. A single GPU rank may be part of a data-parallel group, a tensor-parallel group, an expert-parallel group, and a pipeline stage. Each group has different communication patterns and timing. A programming model should allow different communication mechanisms to coexist without conflicting over streams, buffers, proxy threads, or network resources. If one custom collective monopolizes SMs or queue pairs, it may harm another parallelism dimension. This is why a primitive API alone is not enough; the runtime must coordinate resource use across communication programs.

The DSL layer can also support experimentation. Communication research often requires trying small variants: changing chunk size, changing stage order, changing which dimension communicates first, or changing synchronization placement. If each variant requires writing a new kernel and transport path, experimentation is slow and error-prone. A DSL lowers the cost of exploring variants while still allowing optimized primitive execution. This matters for AI workloads because the best communication algorithm can change when the model architecture, batch size, sequence length, or hardware changes. Programmability is therefore not only about peak performance; it is about reducing the time from workload observation to communication optimization.

However, a programmable interface should avoid making performance unpredictable. If users can write arbitrary communication programs, the runtime must provide guardrails. It should validate that a program does not deadlock under normal execution, that synchronization values are used consistently, and that buffer accesses do not violate declared lifetimes. It should also provide performance diagnostics: which primitive caused delay, which channel was selected, how much time was spent waiting, and whether communication overlapped with computation. These requirements are not fully solved by MSCCL++, but the layered architecture makes them possible because each layer has identifiable responsibilities.

The programming chapter therefore suggests a principle for future APIs: expose intent first, expose schedules second, and expose primitive control only when necessary. Ordinary applications should express intent through collectives. Algorithm designers should express schedules through a DSL. Backend experts should express primitive implementations and channels. This keeps the common path simple while preserving escape hatches for new workloads. The principle is similar to how compilers allow most programmers to write high-level code but still allow

intrinsics or assembly for performance-critical paths. Communication libraries need the same gradient of control.

Chapter 4

Workload and Evaluation: Large-Scale Collective Communication

4.1 Motivation

The workload and evaluation layer asks what communication systems must actually do. Modern LLM workloads are not uniform. Training stresses throughput, startup time, and fault tolerance. Inference stresses low latency and CPU overhead. MoE stresses dynamic routing and AllToAll-style communication. Large clusters stress topology awareness and congestion control. This means evaluation cannot be limited to a single AllReduce microbenchmark.

The three selected papers define complementary workload views. NCCLX focuses on Llama4 training and inference at Meta scale [10]. MSCCL++ evaluates collectives and inference workloads across NVIDIA and AMD hardware [9]. SyCCL evaluates synthesized schedules across collectives, message sizes, and topologies [1]. Together they form a broader benchmark perspective for collective communication systems.

The nonuniformity is visible even inside one training step. A large gradient reduction may be bandwidth-bound and tolerant of a few extra microseconds of launch overhead. A tensor-parallel collective inside a transformer block may be smaller but occur many times per step, so latency and overlap matter. A pipeline send/receive may contend with compute kernels if it uses GPU SMs for copying. An optimizer-sharding collective may be tied to memory pressure. An MoE token exchange may have irregular counts across experts. If these cases are all summarized as "communication overhead," the evaluation loses the information needed to select mechanisms.

NCCLX's workload motivation is the most production-oriented. It explicitly separates generic requirements, training requirements, multi-node inference requirements, control-plane requirements, and tooling requirements. Generic requirements include host-driven customization, zero-copy data transfer, tensor registration management, and network co-design. Training requirements include pipeline-parallel send/receive, tensor-parallel RMA put, and fault-tolerant AllReduce. Inference requirements include GPU-resident collectives and small-message low-latency optimization. This taxonomy is useful because it shows that a single model lifecycle contains several distinct communication problems.

This taxonomy is broader than the benchmark tables common in communication papers. It treats initialization time, metadata movement, failure diagnosis, and resource management as workload properties. For a production LLM cluster, a communication library that is fast only in steady state may still be expensive if it restarts slowly, consumes excessive proxy CPU resources, or provides no way to identify a failed host. NCCLX therefore evaluates not only direct communication latency but also operational consequences. This is consistent with the history of large-scale distributed training systems, where end-to-end productivity depends on throughput, stability, and recovery, not only on kernel speed [5, 7].

MSCCL++'s workload motivation is portability across hardware and algorithms. Its experiments include NVIDIA A100, NVIDIA H100, and AMD MI300x. This matters because the same collective API must perform across different interconnects and runtime stacks. SyCCL's workload motivation is schedule diversity: the same collective can behave differently on 16 GPUs, 32 GPUs, or hundreds of GPUs, and the best schedule can change with message size. These three workload views together justify why the report does not reduce evaluation to one benchmark.

4.2 Task Definition and Benchmark

The communication task can be defined as satisfying data-movement demands among GPUs. In standard collectives, each operation specifies source data, destination requirements, and sometimes reduction semantics. SyCCL formalizes this through participants, chunks, source mappings, destination mappings, and a reduction flag [1]. This model covers one-to-one, one-to-all, all-to-one, and all-to-all collectives.

The value of SyCCL's formal definition is that it separates the semantic demand from the implementation schedule. The semantic demand says which chunks must end up on which GPUs and whether reductions are required. The schedule says how the system will move intermediate data over links and in what order. This separation is important because many schedules can satisfy the same collective semantics. A ring, tree, hierarchical schedule, or synthesized sketch combination can all be correct for the same operation, but their performance differs with topology and size. The formalism creates a common language for comparing those choices.

In real AI systems, the task also includes framework integration. NCCLX operates under PyTorch and manages communication for training and inference. It supports host-initiated APIs, host-initiated APIs with GPU-resident metadata, and device-initiated APIs. MSCCL++ initializes processes, exchanges metadata through a bootstrap interface, constructs communicators, registers buffers, and creates channels. The benchmark is therefore not merely a mathematical collective; it is a communication workload embedded in a deep learning system.

SyCCL's formal task definition is especially precise. A collective contains a set of GPUs, a set of chunks, a chunk size, a mapping from each chunk to its original source GPU, a mapping from each chunk to its destination GPU set, and a flag indicating whether reduction is required. This representation covers SendRecv, Broadcast, Scatter, Gather, Reduce, AllGather, AllReduce, AllToAll, and ReduceScatter. By reducing all these operations to chunk placement and demand satisfaction, SyCCL can synthesize schedules for several collective categories using one framework.

NCCLX's task definition is less mathematical but more operational. The task is to provide reliable, high-throughput, and low-latency data exchange across the Llama4 lifecycle. That means the benchmark includes not only steady-state data movement but also initialization, resource use, and fault localization. For example, scalable initialization is evaluated because frequent restarts make startup time part of effective training throughput. Fault localization is evaluated because a large job that hangs without diagnosis wastes expensive cluster time.

4.3 Four-Dimensional Evaluation

Following the reference thesis's evaluation style, this report views collective communication evaluation along four dimensions. The first dimension is latency, especially for small messages and inference decoding. MSCCL++ reports small-message collective latency, while NCCLX emphasizes low-latency inference paths and GPU-resident metadata. The second dimension is bandwidth, especially for large training collectives. MSCCL++ reports algorithmic bandwidth and SyCCL reports bus bandwidth for synthesized schedules.

The third dimension is scalability. NCCLX evaluates initialization and communication at very large scales, including 96K and 100K+ GPU settings. SyCCL measures synthesis time as topology size grows, showing cases where TECCL takes minutes or hours while SyCCL finishes in

seconds or minutes. The fourth dimension is operability. NCCLX includes resource management, CollTrace, and Fault Analyzer because real training runs fail and must be diagnosed.

The latency dimension includes several different phenomena. In MSCCL++, small-message collectives are evaluated in microseconds. In NCCLX, small-message inference overhead includes CPU preparation cost, CUDA graph constraints, and token routing metadata. In SyCCL, small-message schedule performance is influenced by hop count and link latency. A schedule with too many hops performs poorly even if it uses bandwidth efficiently for larger messages.

Small-message latency has become especially important for inference because decoding is sequential at the token level. Even if the model uses batching, each decoding iteration often performs relatively small communication operations compared with training. If preparing a collective requires CPU-side metadata construction, synchronization with the GPU, or graph-breaking operations, the overhead can be visible in user-facing latency. This is why NCCLX's AllToAllvDynamic focuses on GPU-resident metadata and avoidance of padded transfers. The problem is not simply making one copy faster; it is removing unnecessary host work from the critical path.

The bandwidth dimension is also multi-layered. MSCCL++ reports algorithmic bandwidth for large messages. SyCCL reports bus bandwidth and explains how fixed NCCL schedules can impose bandwidth ratios that mismatch the hardware topology. NCCLX addresses bandwidth at the transport level through zero-copy transfer and DQPLB. These are different ways of measuring and improving the same underlying property: how effectively the system moves useful bytes.

Bandwidth measurement is subtle because reported bandwidth can hide where the bottleneck occurs. A collective may achieve high algorithmic bandwidth while still using the network unevenly. A transport may move bytes quickly while consuming too much GPU memory for staging. A synthesized schedule may improve bus bandwidth but require a runtime that can express its ordering and synchronization. For this reason, the report treats bandwidth as a property of the whole pipeline. The useful metric is not only bytes per second on a link, but useful model progress per unit of communication resource.

Operability is the fourth dimension because modern AI training is not a short experiment. A 100K-GPU training job may restart often due to faults. Initialization overhead, resource pressure, and debugging time can determine the effective cost of training. NCCLX's inclusion of scalable initialization and Fault Analyzer is therefore not peripheral; it is part of the evaluation problem.

4.4 Communication-System Pipeline

The communication-system pipeline begins when the model or framework issues a communication request. In an MSCCL++-like stack, the request may go through the Collective API, the DSL API, or the Primitive API. In a SyCCL-like stack, a collective and topology may first be converted into sketches and sub-demands, synthesized into sub-schedules, simulated, and selected. In an NCCLX-like stack, the selected operation is executed through CTran, using NVLink, InfiniBand/RoCE, or socket backends.

Several concrete mechanisms appear in this pipeline. NCCLX's tensor registration management makes zero-copy transfer practical with PyTorch's caching allocator. DQPLB uses one control QP and multiple data QPs to balance RoCE traffic while limiting outstanding data to avoid congestion. AllToAllvDynamic uses GPU-resident metadata from MoE routing to avoid padded transfers under CUDA graph constraints. These mechanisms show that evaluation must include the full pipeline, not just the final data copy.

The NCCLX copy/RDMA pipeline figure illustrates why zero-copy matters. In a copy-based design, the sender copies from a user buffer into FIFO staging buffers, RDMA transfers from FIFO to remote FIFO, and the receiver copies from FIFO into the user receive buffer. This pipeline can overlap steps, but it consumes GPU memory and SM resources. In pipeline-parallel training, those SM resources may contend with GEMM kernels. NCCLX's zero-copy send/receive avoids the staging copies by allowing RDMA to write directly from the sender user buffer to the receiver user buffer after registration metadata is exchanged.

The SyCCL pipeline is different. It starts from topology and collective demand, searches sketches, generates sketch combinations, synthesizes sub-schedules, simulates complete schedules, and chooses the best candidate. The workload pipeline therefore includes a preparation phase before execution. The output is a schedule that the communication runtime must implement. This is why schedule synthesis and communication programming should eventually connect: a synthesized schedule needs a representation such as a DSL or primitive sequence.

The pipeline view also helps explain why the selected papers are complementary. MSCCL++ mainly improves the representation and execution of communication programs. SyCCL mainly improves the generation of schedules for a given demand and topology. NCCLX mainly improves the production runtime that executes communication under real framework and cluster constraints. A future system would connect these stages into one pipeline: detect communication demand from the model runtime, synthesize or select a schedule, lower it into a programmable communication representation, execute it through a transport-aware runtime, and feed traces back into later decisions.

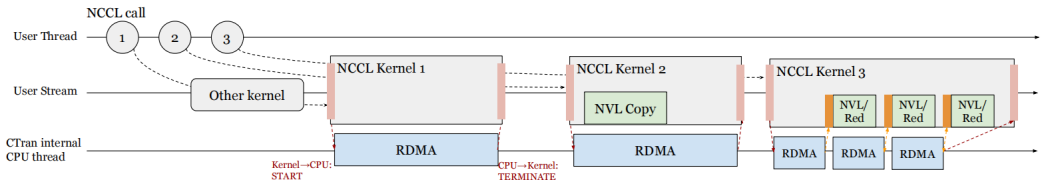


Figure 4.1: Coordination between CTran internal CPU thread and the CUDA kernel for a NCCL collective. [10].

Figure 4.1 makes the evaluation problem concrete by showing the coordination between CTran's internal CPU thread and the CUDA kernel. This coordination is part of the critical path for NCCL-compatible collectives, so it affects not only bandwidth but also CPU overhead, kernel progress, and synchronization cost.

4.5 Results

The papers report significant results at their respective layers. MSCCL++ reports up to 5.4 times speedup for collective communication and up to 15 percent speedup for real-world AI inference workloads [9]. It evaluates A100, H100, and MI300x environments and compares with NCCL, RCCL, and MSCCL. The key result is that the layered abstraction does not prevent high performance.

SyCCL reports up to 91 percent improvement over TECCL and 108 percent over NCCL in a 32-A100 setting, up to 127 percent improvement in production-scale H800 simulation, and up to 6.3 percent end-to-end model training improvement [1]. It also reports synthesis-time reductions from 915 times to 17,286 times in several scenarios. NCCLX reports reduced Llama4 training step latency, improved Llama4 Maverick decoding latency, and up to 11 times faster initialization over baseline NCCL at large scale [10].

The MSCCL++ collective results are nuanced. For AllReduce on A100, the paper reports large gains for small messages and smaller but still meaningful gains for large messages. It explains that different algorithms win at different sizes. For very small messages, 1PA can avoid synchronization overhead. For larger messages, two-phase and hierarchical algorithms reduce redundant traffic. This reinforces the need for a programming model that can hold multiple algorithms.

The practical lesson is that a communication system should not be evaluated only at the message size where it looks best. Training and inference systems sweep across sizes constantly. Gradients, activations, router metadata, expert outputs, and optimizer shards have different shapes. A model using mixed precision, activation checkpointing, tensor parallelism, and expert parallelism can change the communication-size distribution when the batch size or sequence

length changes. An evaluation that covers only one collective and one size would therefore hide the real deployment risk. The selected papers avoid this by reporting across operations, sizes, topologies, and system contexts.

SyCCL's results are also nuanced by collective type. For AllGather, it benefits from avoiding excessive hops and better matching topology bandwidth ratios. For ReduceScatter, gains can be smaller because existing libraries may have reduction-specific optimizations. For synthesis time, the paper shows that sketch search and combination generation are not the main bottlenecks; most time is spent solving sub-schedules. This supports the design claim that SyCCL's symmetry handling removes large parts of the search space.

NCCLX's results should be read as production evidence. It reports improvements in training step latency and inference decoding latency, but it also reports initialization speedups and tooling improvements. The initialization result is particularly important: baseline initialization becomes problematic near 100K GPUs due to serialized operations, network bottlenecks, and bootstrap complexity. NCCLX optimizes the control path across PyTorch, NCCL, and CTran, making restarts less costly.

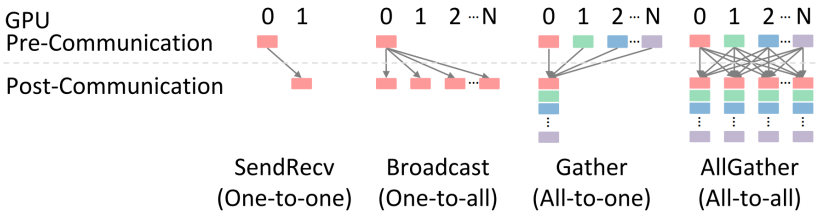


Figure 4.2: Collectives are categorized into four types: one-to-one, one-to-all, all-to-one, and all-to-all.

Figure 4.2 is useful because it shows why SyCCL can discuss several collectives with one formalism. Instead of treating AllGather, AllToAll, ReduceScatter, and AllReduce as unrelated routines, SyCCL represents them through common source, destination, chunk, and reduction semantics.

4.6 Summary

This chapter defined the workload and evaluation layer. The key lesson is that collective communication must be evaluated as part of model execution and cluster operation. Latency, bandwidth, scalability, and operability all matter. This prepares the runtime discussion in the next chapter, where these workload requirements are mapped to concrete scheduling and transport mechanisms.

The workload chapter also explains why the three papers report different types of results. MSCCL++ is interested in whether its abstraction can produce fast collective kernels; therefore, it reports latency and bandwidth across message sizes and hardware platforms. SyCCL is interested in whether schedule synthesis can be made practical; therefore, it reports schedule performance and synthesis time. NCCLX is interested in whether a production communication stack helps a real model lifecycle; therefore, it reports training, inference, initialization, and fault-localization-oriented results.

These differences should be preserved rather than collapsed. If we judged SyCCL only by end-to-end training speed, we might miss the significance of reducing synthesis time by orders of magnitude. If we judged NCCLX only by a single collective bandwidth number, we might miss the importance of scalable initialization and Fault Analyzer. If we judged MSCCL++ only by one API layer, we might miss the value of offering three different programming interfaces. A systems evaluation must match the system boundary.

The four-dimensional evaluation view also reveals hidden costs. Padding in inference may not appear in a traditional collective benchmark, but it increases user-facing latency in MoE serving. Memory registration spikes may not appear in steady-state bandwidth, but they can

hurt dynamic workloads. A schedule synthesis process may not affect per-iteration runtime once the schedule is generated, but if it takes hours, it cannot support practical deployment. These costs motivate the detailed runtime mechanisms discussed in Chapter 5.

Another lesson is that figures and tables are not merely illustrative. The NCCLX pipeline figure makes the cost of staging copies concrete. The SyCCL collective-category figure shows why one formal model can cover multiple communication patterns. The SyCCL workflow figure shows the separation between sketch exploration and schedule synthesis. The DQPLB figure shows that network load balancing is implemented through concrete queue-pair mechanisms. These diagrams help turn abstract communication claims into system structure.

The chapter answers the second research question by showing that the workload is multi-phase and multi-metric. A collective communication system for LLMs must serve training, prefill, decoding, expert routing, initialization, and recovery. It must perform well on small and large messages, intra-node and inter-node links, static and dynamic communication patterns, and regular and irregular topologies. This workload complexity is exactly why the runtime layer needs multiple mechanisms rather than one fixed schedule or one fixed API.

A further workload lesson is that benchmark selection encodes assumptions about the system boundary. If a paper evaluates only steady-state collective bandwidth, it implicitly treats initialization, metadata movement, and debugging as outside the communication system. NCCLX rejects that boundary by including scalable initialization and fault analysis [10]. If a paper evaluates only a fixed set of library collectives, it implicitly treats custom algorithms as outside the benchmark. MSCCL++ rejects that boundary by evaluating primitive and DSL-based implementations [9]. If a paper evaluates only execution time and ignores schedule-generation time, it implicitly assumes schedules appear for free. SyCCL rejects that boundary by measuring synthesis time as a primary result [1].

The workload layer also forces a distinction between static and dynamic communication. Static collectives have known participants, sizes, and destinations before execution. Many training collectives fall into this category. Dynamic communication has sizes or destinations determined by runtime computation. MoE routing is the most important example in the selected papers, because the router kernel decides which tokens go to which experts. Dynamic communication is harder to optimize because the runtime cannot precompute all metadata on the CPU without adding latency or breaking graph capture. NCCLX's AllToAllvDynamic directly targets this case by allowing GPU-resident metadata to drive the operation.

Another dimension is reuse. Some communication patterns repeat thousands or millions of times during training. For these, it is acceptable to spend more time on initialization, schedule synthesis, or tuning if the result is reused. Other patterns change frequently, especially in dynamic inference. For these, the per-operation overhead must be extremely small and the runtime must avoid expensive CPU work. This difference affects how one should evaluate SyCCL-style synthesis. A synthesis process that takes seconds may be excellent for a long training job but inappropriate for per-request inference adaptation. The system must know when a schedule is reusable.

The selected papers also differ in how they treat hardware diversity. MSCCL++ explicitly evaluates portability across NVIDIA and AMD environments, showing that a primitive abstraction can reduce backend-specific code while still achieving high performance [9]. SyCCL evaluates different topologies and scales, showing that schedule quality depends on the physical arrangement of GPUs and links [1]. NCCLX evaluates production requirements in Meta's environment, showing that large-scale deployment exposes bottlenecks that may not appear in smaller laboratory settings [10]. Together, these evaluations show that hardware diversity is not only a portability problem but also a workload-definition problem.

A complete benchmark suite would therefore need to cover several axes. It should include operation type, message size, topology, scale, model phase, hardware platform, graph-capture mode, metadata location, and failure condition. It should measure latency, bandwidth, CPU usage, GPU memory overhead, SM usage, initialization time, schedule generation time, and diagnosis time. This sounds broad, but the selected papers show why each dimension matters. A system can win on bandwidth and lose on CPU overhead; win on synthesis quality and lose

on synthesis time; win on one topology and lose on another; win in microbenchmarks and lose in a full model due to resource contention.

This workload view also explains why end-to-end model results are valuable but not sufficient. End-to-end speedup is the most convincing evidence that a communication mechanism matters, but it can be hard to interpret. A small end-to-end improvement may hide a large communication improvement if communication is only a fraction of total time. A large communication improvement may not translate to model speedup if the workload is compute-bound. Microbenchmarks isolate mechanisms, while end-to-end results show practical impact. The selected papers use both styles, and the report treats them as complementary rather than competing evidence.

Finally, the workload chapter shows that communication evaluation has an operational cost dimension. A cluster operator cares about how quickly jobs start, how many resources communication consumes, how easily failures are diagnosed, and whether a new communication mechanism is safe to deploy. These concerns are rarely visible in simple latency/bandwidth plots, but they strongly affect real training cost. NCCLX's inclusion of control-plane and tooling requirements is therefore one of its most important contributions. It broadens the benchmark from "how fast is a collective" to "how well does the communication stack support a large AI job over its full lifecycle."

The workload view also makes clear that different communication phases have different tolerance for complexity. A long-running training job can amortize expensive schedule synthesis, communicator setup, and buffer registration over many iterations. In contrast, an online inference service has less tolerance for per-request setup and may prioritize predictable tail latency over peak bandwidth. This means the same communication mechanism may be appropriate in one phase and inappropriate in another. A SyCCL-generated schedule may be excellent for repeated training collectives, while a simpler GPU-resident dynamic collective may be better for decode-time MoE traffic. Evaluation should therefore associate each mechanism with the phase where it is expected to help.

Another important evaluation issue is tail behavior. Average latency and average bandwidth are useful, but production systems also care about variance and outliers. A rare registration spike, delayed proxy thread, or congested queue pair can increase step time or serving latency. NCCLX's discussion of tensor registration spikes and fault localization shows that outliers are not theoretical concerns [10]. In synchronous training, the slowest rank can determine step time. In serving, tail latency affects user experience. A complete evaluation should therefore include not only mean performance but also the causes of variance.

The evaluation layer should also account for CPU overhead. GPU clusters are expensive because GPUs are the scarce resource, but CPU overhead can still limit communication progress. CPU proxy threads may process requests, post RDMA operations, or prepare metadata. If small messages require frequent CPU involvement, the CPU can become a bottleneck even when the network is underused. MSCCL++'s PortChannel explicitly includes a CPU thread in the path for some transfers, while NCCLX tries to move certain metadata and initiation paths toward the GPU [9, 10]. Measuring only GPU-visible latency can miss this resource cost.

Memory overhead is another hidden evaluation dimension. Staging buffers, FIFO queues, registered segments, synchronization variables, and temporary reduction buffers consume GPU memory. In LLM training, memory is already constrained by parameters, activations, optimizer state, and KV cache for inference. A communication algorithm that improves bandwidth by using large temporary buffers may not be acceptable if it reduces maximum batch size or sequence length. NCCLX's zero-copy work partly addresses this by avoiding extra staging copies, but zero-copy introduces registration complexity. This is a typical systems trade-off: saving one resource often increases pressure on another.

Evaluation should also distinguish between library-level speedup and model-level speedup. A 2x speedup for one collective does not imply a 2x model speedup unless that collective dominates the critical path. Conversely, a modest communication improvement can be valuable if it affects a highly repeated operation. MSCCL++ reports both communication microbenchmarks and inference speedups [9]; SyCCL reports schedule performance and end-to-end model training improvement [1]; NCCLX reports training, inference, and initialization results [10]. This mix is

necessary because no single metric captures the whole value of a communication system.

The workload chapter also implies that benchmark reproducibility is challenging. Large clusters are difficult to reproduce across institutions. Production networks may have private topologies and operational constraints. Hardware generations differ in interconnect capabilities. Model workloads evolve quickly. A useful paper must therefore provide enough formalism, diagrams, and methodology that readers can transfer the insight even if they cannot reproduce the exact cluster. SyCCL's formal task definition, MSCCL++'s API hierarchy, and NCCLX's requirement taxonomy all help in this respect. They make the contribution understandable beyond the exact experimental environment.

Chapter 5

Runtime: Dynamic Collective Scheduling

5.1 Motivation

The runtime layer asks how communication is executed efficiently after it has been expressed and evaluated. A runtime must choose transports, schedule data movement, overlap communication with computation, manage resources, and handle failures. The challenge is that the best runtime behavior depends on topology, message size, model phase, and hardware condition.

This chapter uses the word runtime broadly. It includes the online data path that transfers bytes, but it also includes initialization, resource allocation, schedule preparation, metadata management, and tracing. This broad definition is necessary because communication failures and overheads often occur outside the obvious data-copy instruction. A job can stall because a communicator was initialized incorrectly, a memory region was not registered, a proxy thread is overloaded, a queue pair is congested, or a dependency between collectives hides the original failure. Runtime design therefore includes both performance engineering and operational engineering.

NCCLX and SyCCL address this layer from different angles. NCCLX builds a production runtime around CTran, zero-copy transfer, DQPLB, scalable initialization, and fault localization. SyCCL builds a schedule synthesis runtime that searches sketches, solves sub-schedules, simulates candidate schedules, and chooses the best one. Together they show how runtime design moves beyond fixed rings and host-only collectives.

The runtime layer is where the difference between a paper algorithm and a deployable system becomes most visible. A collective algorithm can be elegant, but it still needs memory registration, transport selection, synchronization, congestion control, and fault handling. Conversely, a production transport can be robust, but it still needs good schedules and expressive APIs. This chapter therefore combines NCCLX's production runtime mechanisms with SyCCL's scheduling runtime, using MSCCL++ as the likely execution substrate for programmable algorithms.

The key runtime question is when decisions are made. Some decisions are made before execution, such as SyCCL schedule synthesis. Some are made during initialization, such as NCCLX communicator setup and resource allocation. Some are made during execution, such as DQPLB controlling outstanding messages or AllToAllvDynamic using GPU-resident routing metadata. A complete system must coordinate decisions across all these time scales.

5.2 Architecture Overview

NCCLX's architecture operates beneath PyTorch and manages both training and inference communication. It provides multiple execution modes and communication semantics, and CTran supplies the transport layer. CTran supports NVLink, InfiniBand/RoCE, and socket backends,

as well as custom collectives, RMA, and zero-copy point-to-point transfer. Its design principle is to promote zero-copy and SM-free communication when possible.

SyCCL's architecture has two phases. First, sketch exploration generates potential decompositions of the collective demand according to topology and collective symmetry. Second, schedule synthesis produces sub-schedules for the generated sketch combinations, merges them into complete schedules, simulates their performance, and selects the best. This architecture separates the search for promising structure from the detailed synthesis of data movement.

CTran is the central NCCLX runtime component. It supports multiple communication semantics and backends. It extends baseline NCCL in several ways: supporting several execution models through one framework, supporting standard and custom collectives as well as RMA, providing NVLink, InfiniBand/RoCE, and socket backends, and optimizing the critical path for low-latency small-message scenarios. Its design principle is to use zero-copy and SM-free communication when possible.

This design is especially relevant for pipeline-parallel and tensor-parallel workloads. In pipeline parallelism, send/receive traffic can be adjacent to large compute kernels. If communication requires GPU SMs for copying, it may interfere with GEMM throughput. In tensor parallelism, fine-grained RMA-style updates can benefit from lower overhead and better overlap. CTran's support for multiple semantics lets NCCLX choose different mechanisms for these different phases. This is a runtime-level analogue of MSCCL++'s programming hierarchy: one system supports several execution modes instead of forcing every workload through one path.

DQPLB is one concrete CTran mechanism. It uses a control QP to exchange memory addresses and multiple data QPs to carry data. It limits the number of data QPs, outstanding messages per QP, and segment size according to connection type. Same-rack connections use more conservative settings because the bandwidth-delay product is smaller; distant connections can use more aggressive settings. Sequence numbering and out-of-order tracking preserve ordered delivery while allowing dynamic load balancing across QPs.

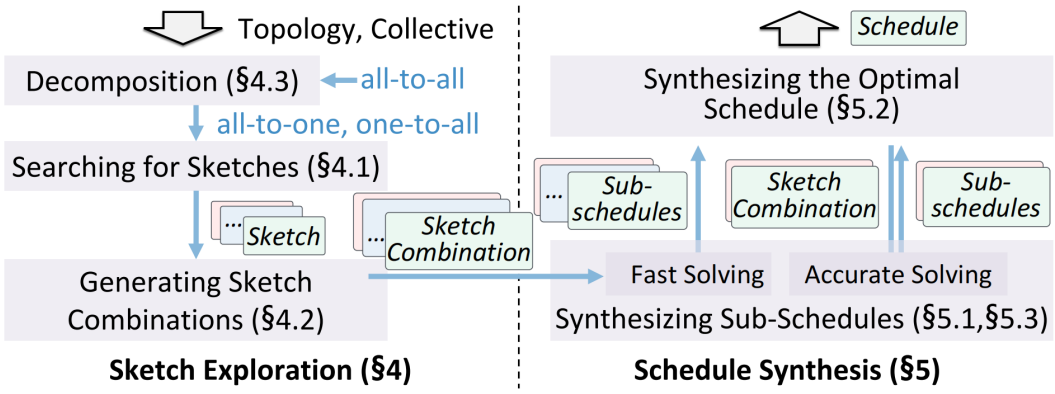


Figure 5.1: SyCCL workflow.

Figure 5.1 highlights the staged nature of SyCCL synthesis. The system first reduces the search space through sketch exploration and only then applies more expensive sub-schedule synthesis and simulation. This is the scheduling analogue of the layered programming design in MSCCL++.

5.3 Formalisms and Scheduling Algorithm

SyCCL formalizes a collective through GPU participants, chunks, source mappings, destination mappings, and reduction semantics. It then introduces a sketch: a multi-stage decomposition of the original demand into sub-demands over dimensions and groups. For one-to-all and all-to-one collectives, schedules follow tree structures. For all-to-all collectives, SyCCL decomposes them into isomorphic one-to-all collectives and replicates sketches according to topology symmetry.

After sketch combinations are generated, SyCCL merges sub-demands that occur in the same GPU group and stage. It models each merged sub-demand as a MILP problem using an alpha-beta transmission model. A transmission of size s over a link has a latency component α and bandwidth component βs. Time is divided into epochs so integer variables can represent communication events. After sub-schedules are synthesized, SyCCL uses an ASTRA-sim-based simulator to account for overlap across stages and choose the best complete schedule.

NCCLX's runtime algorithms are transport-oriented. Tensor registration management caches registered CUDA segments to make zero-copy feasible. DQPLB distributes messages across data QPs while controlling outstanding messages by connection type. In pipeline parallelism, zero-copy and SM-free send/receive avoid staging copies and reduce contention with GEMM kernels. In tensor parallelism, RMA put supports fine-grained overlap. In inference, AllToAllvDynamic uses GPU-resident metadata to avoid padded transfers.

SyCCL's sketch formalism gives the scheduling algorithm its structure. A sketch has stages, and each stage contains sub-demands in topology dimensions and groups. In one-to-all communication, the schedule follows a tree because each destination should receive the chunk only once. In all-to-all communication, SyCCL decomposes the collective into isomorphic one-to-all collectives. For large chunks, a single sketch may underutilize bandwidth, so SyCCL generates sketch combinations, each assigning a fraction of the chunk to a sketch. This allows multiple paths to share traffic.

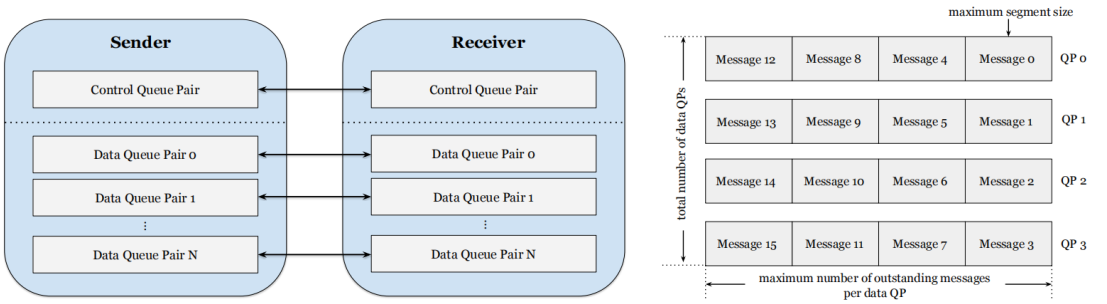
The key insight is that symmetry reduces both reasoning cost and solver cost. Many GPU clusters contain repeated groups: GPUs within a server, servers within a rack, racks within a cluster, or logical dimensions in a torus-like topology. Many collective demands also contain repeated structure: every GPU sends a similar kind of data or every chunk has a similar destination pattern. SyCCL exploits these repeated structures so that it does not need to synthesize every transmission independently. Instead, it can solve representative sub-problems and replicate or combine them according to topology symmetry. This is why the approach scales better than a flat optimization over the full schedule space.

For sub-schedule synthesis, SyCCL models each merged sub-demand as a MILP problem using the alpha-beta model. The solver divides time into epochs and chooses transmissions that satisfy the sub-demand with minimum completion time. After sub-schedules are merged, a simple sum of stage durations would be inaccurate because later stages can begin as soon as their source GPUs receive data. SyCCL therefore uses an ASTRA-sim-based event simulator to compute candidate schedule completion times more accurately.

NCCLX's runtime algorithms include tensor registration management. Zero-copy requires user buffers to be registered for NVLink or network direct transfer. Registration can be expensive and can suffer from spikes due to driver lock contention. NCCLX co-designs with PyTorch's CUDA caching allocator so that CUDA segments can be tracked and cached. This makes zero-copy feasible in a framework where tensor lifetimes are normally implicit.

The registration problem shows why framework integration matters. A standalone communication benchmark can allocate fixed buffers, register them once, and reuse them. A real PyTorch workload allocates and frees tensors through a caching allocator, reuses memory segments, and may change tensor lifetimes across iterations. If the communication library registers every tensor naively, it can create expensive registration spikes and synchronization overhead. By aligning registration with allocator segments, NCCLX moves from tensor-by-tensor registration to segment-aware caching. This is a concrete example of cross-layer co-design between a machine learning framework and a communication runtime.

Figure 5.2 shows why load balancing in a production communication runtime is a transport-level concern, not only an algorithm-level concern. The collective algorithm may decide which GPU should send to which GPU, but CTran still has to decide how packets are distributed across queue pairs and how much outstanding traffic is safe for a given connection.



(a) Queue pair framework. **(b)** Example of load balancing across four QPs.

Figure 5.2: NCCLX DQPLB design overview [10].

5.4 Results

The runtime results show benefits at different scales. SyCCL's synthesis is much faster than TECCL because it solves smaller sub-demands and reuses isomorphic schedules. The paper reports 0.3–2.6 seconds for 16-GPU AllGather and 6.2–19.1 seconds for 32-GPU AllGather in cases where TECCL takes minutes to hours [1]. NCCLX's scalable initialization reduces startup time by optimizing PyTorch, NCCL, and CTran control paths, with reported improvements up to 11 times over baseline NCCL [10].

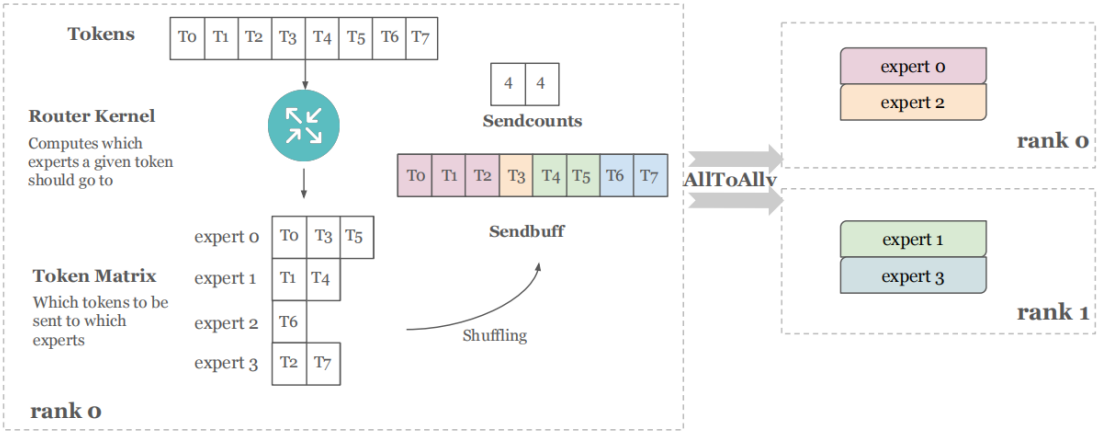


Figure 5.3: NCCLX token shuffling

Figures 5.3 and 5.4 connect the runtime discussion to MoE inference. The relevant metadata is not a static host-side array prepared before the graph runs; it is produced by GPU-side routing and can vary by batch and decoding step. AllToAllvDynamic therefore matters because it lets the kernel consume updated GPU-resident metadata by reference rather than forcing padded host-side preparation.

The runtime results also include production-specific outcomes. NCCLX's Fault Analyzer uses CollTrace and collective interdependency-aware rules to identify the first stalled collective and localize faulty hosts. MSCCL++'s runtime support appears through the DSL Executor and channel implementations, which allow algorithms to run through portable primitive mechanisms while still using hardware-specific paths.

Fault localization is difficult in multi-dimensional parallelism because a single physical failure can cause many logical collectives to stall. A rank may participate in data-parallel, tensor-parallel, pipeline-parallel, and expert-parallel groups. If one group stalls, other groups may later stall because their kernels depend on earlier results. A naive diagnosis may report the last visible hang rather than the first cause. NCCLX's use of collective traces and dependency-aware rules addresses exactly this problem: it attempts to reconstruct which collective first stopped

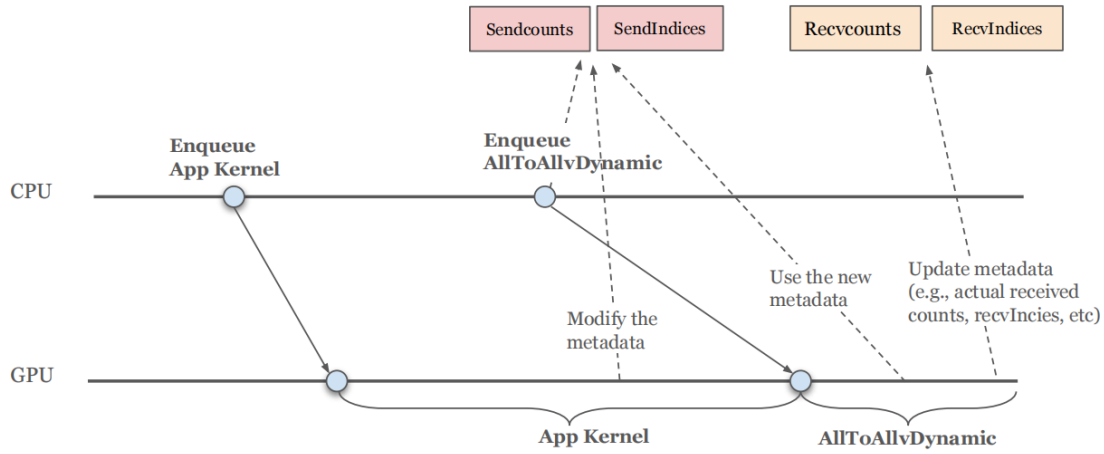


Figure 5.4: NCCLX AlltoAllvDynamic implementation. AlltoAllvDynamic takes the metadata by reference, allowing its kernel to use the new metadata (e.g., send counts) and updates other metadata (e.g., recv counts) that are needed by the users.

making progress and which host is likely responsible. This is a communication-layer analogue of distributed tracing in service systems.

NCCLX's AllToAllvDynamic is a runtime result for inference. Traditional AllToAllv takes host-side metadata such as send counts and buffer addresses. In MoE inference, this metadata is produced by a GPU router kernel. If the runtime must pad data to fit static CUDA graph assumptions, it sends unnecessary bytes. GPU-resident collectives avoid this by using the routing metadata on the GPU and transferring only the actual token data needed by experts. This directly targets decoding latency.

NCCLX's scalable initialization is another runtime result. During process-group creation, all ranks must exchange metadata and allocate resources. At small scales, this may be acceptable; near 100K GPUs, serialized bootstrap operations and network bottlenecks can dominate restart time. NCCLX optimizes the PyTorch, NCCL, and CTran control path, reporting up to 11 times faster initialization. This result is not a microbenchmark result, but it affects effective training productivity.

Fault Analyzer adds a reliability result. NCCL's RAS subsystem helps diagnose crashes and hangs, but NCCLX identifies two limitations: cascaded failures across communicators in multi-dimensional parallelism and insufficient kernel start/end tracking. NCCLX adds tracing at per-collective and RDMA granularity through CollTrace and uses dependency-aware rules to locate the original stalled collective and faulty host. This is essential when manual debugging can take hours or days.

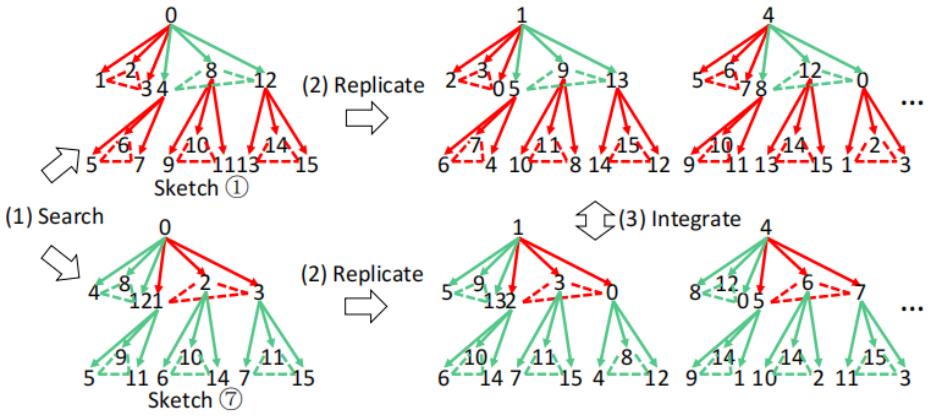


Figure 5.5: Example of generating AllGather SyCCL sketches.

Figure 5.5 illustrates the idea that a schedule can be reasoned about as a structured decomposition rather than as a flat list of transmissions. This is the key reason SyCCL can reuse symmetry and avoid solving one enormous monolithic scheduling problem.

5.5 Discussion

The runtime layer exposes several trade-offs. A host-driven framework is easier to integrate but may create CPU overhead for small messages. A device-initiated or GPU-resident path can reduce overhead but complicates synchronization and memory visibility. A synthesized schedule can improve topology utilization but must be expressed and executed by a communication library. A zero-copy path can reduce copies but requires careful memory registration and lifetime management.

The systems therefore need layered fallbacks. NCCLX keeps host-initiated APIs while extending toward GPU-resident metadata and device initiation. MSCCL++ keeps a Collective API while exposing DSL and primitive layers. SyCCL reduces search through symmetry but still uses MILP and simulation internally. In each case, the runtime is not one mechanism but a coordinated set of mechanisms.

The most important runtime trade-off is between specialization and manageability. Specialized mechanisms such as zero-copy transfer, GPU-resident collectives, and synthesized schedules can improve performance, but they also introduce complexity. Buffer registration must be correct, synchronization must be safe, generated schedules must be executable, and failures must be diagnosable. NCCLX addresses manageability through tooling, while MSCCL++ addresses it through API layers, and SyCCL addresses it through decomposition.

Specialization also creates versioning and portability challenges. A schedule generated for one topology may not be optimal on another topology. A SwitchChannel algorithm may depend on NVLink 4.0 multimem support and therefore not transfer directly to another platform. A GPU-resident inference collective may depend on how the model runtime represents routing metadata. The runtime must therefore distinguish between semantic compatibility and performance compatibility. An operation may be semantically portable while still requiring different algorithms, channels, or schedules for good performance.

Another trade-off is between offline and online optimization. SyCCL can spend time synthesizing schedules because schedules may be reused across long jobs. NCCLX must make some decisions online, such as controlling outstanding data in DQPLB or handling dynamic inference metadata. A future system may combine both: precompute schedules for common collectives, but adapt transport choices and message limits at runtime based on observed network conditions.

5.6 Summary

This chapter described the runtime layer corresponding to the reference thesis's scheduling chapter. The key lesson is that efficient communication requires both transport mechanisms and scheduling intelligence. NCCLX shows the production runtime path; SyCCL shows the schedule synthesis path; MSCCL++ provides the programmable execution substrate that could connect them.

The runtime layer also shows why communication optimization has to be staged. Before execution, the system can inspect topology, message sizes, and collective structure. This is where SyCCL-style sketch search and MILP synthesis are useful. During initialization, the system can exchange metadata, register memory, and allocate transport resources. This is where NCCLX's scalable initialization and tensor registration management are useful. During execution, the system can select QPs, control outstanding messages, use GPU-resident metadata, and overlap communication with computation. This is where CTran, DQPLB, and AllToAllvDynamic are useful.

The staging matters because each optimization has a different time budget. Schedule synthesis can take seconds or minutes if the schedule is reused across a long job. Initialization should

be fast enough that restarts do not dominate training productivity. Per-operation runtime overhead must be extremely small, especially in decoding. A well-designed system places expensive reasoning in earlier phases and leaves the critical path as lean as possible.

NCCLX demonstrates this principle in several places. Tensor registration can be expensive, so NCCLX caches registration information through PyTorch allocator integration. Bootstrap can serialize at large scale, so NCCLX optimizes initialization across PyTorch, NCCL, and CTran. CPU overhead can dominate MoE inference, so NCCLX moves metadata into GPU-resident paths. Network congestion can hurt large flows, so DQPLB limits outstanding traffic based on connection type.

SyCCL demonstrates the same principle for schedule synthesis. It does not ask a MILP solver to solve the entire topology and collective at once. It first searches sketches using symmetry, then solves smaller sub-demands, then simulates candidate schedules. This moves structural pruning before expensive solving. It is a systems design pattern: reduce the problem with domain knowledge before applying a general optimizer.

MSCCL++ provides the missing execution side for such algorithms. A synthesized schedule or hand-designed communication pattern still needs to be lowered into operations that GPUs and transports can execute. The Primitive API and DSL Executor provide such a lowering target. Therefore, the runtime chapter should not be read as only NCCLX plus SyCCL; it depends on the programming layer from Chapter 3.

The main limitation is that these pieces are not yet unified. NCCLX is a production framework, MSCCL++ is a programming abstraction, and SyCCL is a synthesizer. A future runtime would need to connect their interfaces: schedule output to DSL input, DSL primitives to transport backends, and transport telemetry back to schedule selection. This is the cross-layer problem taken up in Chapter 6.

A further runtime issue is resource accounting. Communication consumes more than network bandwidth. It consumes GPU memory for buffers and FIFOs, CPU threads for proxy or PortChannel handling, network queue pairs, registered memory regions, and sometimes GPU SMs. NCCLX explicitly treats resource management as a tooling concern because excessive communication resource usage can reduce resources available to the model. MSCCL++ exposes this issue through channel implementations, and SyCCL exposes it indirectly through schedule choices that determine which links and groups are active.

Another runtime issue is ordering and completion. One-sided and asynchronous communication require careful reasoning about when data is visible and when a receiver can safely consume it. MSCCL++ primitives such as signal, wait, and flush exist to express these events. NCCLX's GPU-resident and device-initiated paths must also respect memory ordering and completion semantics. SyCCL schedules must respect dependency constraints so that a GPU forwards a chunk only after receiving it. These correctness constraints are as important as bandwidth.

The chapter answers the third research question: efficient runtime execution requires topology-aware scheduling, transport-aware data movement, and operation-aware synchronization. SyCCL contributes the scheduling intelligence, NCCLX contributes the production transport and operational runtime, and MSCCL++ contributes a programmable execution substrate. No single mechanism covers all runtime needs.

The runtime layer also shows why communication optimization is partly a resource-allocation problem. Network bandwidth is the obvious resource, but it is not the only one. RDMA queue pairs, completion queues, proxy CPU threads, registered memory regions, staging buffers, GPU SMs, CUDA streams, and synchronization variables are all finite. Increasing the number of data QPs may improve load balancing but also increase resource pressure. Using staging buffers may improve overlap in some cases but consume GPU memory and copy bandwidth. Using GPU SMs for communication may reduce latency but interfere with compute kernels. Runtime design is therefore about choosing the right resource mix for each operation.

DQPLB is a good example of this resource-allocation perspective. It does not simply open as many connections as possible. It uses a control QP and a limited number of data QPs, and it controls outstanding messages and segment sizes according to connection type [10]. This is more subtle than generic load balancing because the network fabric has different bandwidth-delay

behavior across same-rack and cross-rack paths. Too little parallelism underutilizes bandwidth. Too much outstanding traffic can create congestion and out-of-order pressure. The runtime has to balance these effects continuously.

SyCCL shows a different resource-allocation problem: search resources. A naive MILP formulation for a full collective schedule can become too large to solve. SyCCL spends search effort where it is most valuable by using sketches and symmetry to restrict the search space [1]. It then solves smaller sub-schedules and simulates combinations. This is runtime preparation rather than online execution, but it follows the same principle as DQPLB: do not use the most expensive resource blindly. For NCCLX the scarce resource may be network capacity or queue pairs; for SyCCL it is solver time and schedule-search complexity.

The runtime chapter also reveals a tension between static planning and dynamic adaptation. Static planning is powerful when the topology and communication demand are known. SyCCL can exploit this by generating a high-quality schedule before execution. Dynamic adaptation is necessary when runtime conditions change, such as congestion, failures, or MoE routing decisions. NCCLX handles some dynamic behavior through DQPLB and GPU-resident metadata. A future runtime should combine both: use static synthesis for stable collectives, but allow online adjustment for congestion, failures, and dynamic message sizes.

Correctness remains a central runtime issue. A schedule is not correct merely because it eventually sends the right bytes; it must respect dependencies, reductions, buffer lifetimes, and memory visibility. A GPU cannot forward a chunk before it receives it. A receiver cannot consume a buffer before the transfer is complete. A sender cannot reuse a buffer while a zero-copy operation is still reading it. A reduction must combine the correct values exactly once. These constraints appear in different forms across the three papers. SyCCL encodes dependencies in schedule synthesis. MSCCL++ exposes signal, wait, and flush. NCCLX manages registered buffers and traces collective progress.

The runtime layer is also where observability must be designed in rather than added later. If communication becomes programmable and synthesized, debugging becomes harder. A hang may be caused by an incorrect DSL program, a transport failure, a missing signal, a schedule dependency bug, a registration problem, or a network issue. NCCLX's CollTrace is important because it records communication progress at a level that can be related back to collectives and RDMA operations [10]. A future programmable communication stack would need similar tracing across DSL instructions, primitive operations, channel backends, and transport events.

Another runtime concern is interaction with CUDA graphs and model execution scheduling. In inference, CUDA graph capture reduces CPU overhead by replaying a fixed graph, but dynamic communication metadata can conflict with fixed graph assumptions. Padding data to fit a static graph preserves graph capture but wastes bandwidth and increases latency. NCCLX's GPU-resident collective design addresses this by letting metadata produced on the GPU guide communication without round-tripping through the CPU. This example shows that runtime design must understand not only the network but also the execution model of the deep learning framework.

Finally, the runtime layer emphasizes that production communication systems need graceful fallbacks. Not every operation can use the most advanced path. Some platforms may not support a given channel. Some buffers may not be registered. Some schedules may not be synthesized in time. Some dynamic workloads may not match a precomputed plan. A robust runtime should fall back to standard collectives or simpler transport paths while preserving correctness. Performance specialization is valuable only if the system remains usable when the specialized path is unavailable.

Runtime policy is therefore as important as runtime mechanism. Mechanisms provide options: zero-copy transfer, staged-copy transfer, host initiation, GPU-resident metadata, device initiation, ring schedules, synthesized schedules, and primitive kernels. Policy decides which option to use under which condition. A simple policy may select based only on message size. A better policy may also consider topology, graph capture, buffer registration state, model phase, and recent congestion. A production policy must also be conservative enough to avoid unsafe choices. NCCLX's multiple execution modes and SyCCL's schedule-selection process both point

toward this need for explicit policy [10, 1].

The runtime must also manage overlap carefully. Overlap is often presented as a pure benefit: hide communication behind computation. In practice, overlap consumes resources. If communication uses SMs, it may slow the compute kernel it overlaps with. If it uses memory bandwidth, it may interfere with tensor loads and stores. If it uses proxy CPU threads, it may compete with other runtime work. A zero-copy, SM-free path is attractive because it reduces some overlap costs, but it requires registered memory and transport support. The best runtime decision is therefore not simply "overlap whenever possible," but "overlap when the resource interference is smaller than the hidden latency."

Another runtime issue is ordering across multiple communicators. LLM training often uses several process groups. Operations in one group may depend on results from another group, even if the communication library sees them as independent collectives. If a runtime schedules them without regard to these dependencies, it may create avoidable stalls or make failures harder to diagnose. NCCLX's Fault Analyzer addresses the diagnosis side by considering collective interdependencies [10]. A future runtime could also use dependency information proactively for scheduling and resource allocation.

Schedule synthesis also raises runtime deployment questions. A synthesized schedule may assume a certain chunking strategy, link cost, and topology. At runtime, the actual job may run with slightly different message sizes, degraded links, or different rank placement. The runtime must decide whether to reuse the schedule, adjust it, or fall back. This suggests that schedule objects should carry validity conditions: topology assumptions, message-size range, supported transports, and expected cost parameters. Without such metadata, a schedule repository could become unsafe or ineffective as cluster conditions change.

Finally, runtime observability should be tied to user-facing abstractions. A trace that reports only low-level RDMA events may be too hard for model developers to interpret. A trace that reports only high-level collective names may be too vague for runtime engineers. The useful trace connects levels: model operation, collective, schedule stage, primitive instruction, channel, transport event, and hardware counter. This is the kind of cross-layer observability that would make programmable and synthesized communication practical. NCCLX's CollTrace is a step in this direction, and future MSCCL++/SyCCL integrations would need comparable traceability across their own layers.

Chapter 6

Cross-Layer Synthesis and Open Challenges

6.1 Connecting the Layers

The three layers connect naturally. MSCCL++ provides the programming interface through primitives, DSLs, and collectives. SyCCL provides workload-specific schedule synthesis and evaluation. NCCLX provides runtime execution, transport integration, initialization, and fault analysis. A future system could use all three: a framework expresses communication through high-level operations, a synthesizer generates a topology-aware schedule, a DSL or primitive layer represents it, and a production transport executes it.

This cross-layer view also explains why isolated optimization is insufficient. If the programming layer cannot express a schedule, synthesis is not useful. If the runtime cannot execute zero-copy or GPU-resident metadata, the programming abstraction is limited. If evaluation ignores initialization and failure recovery, a fast kernel may not improve real training efficiency. The layers must therefore be co-designed.

The co-design problem can be stated as an information-flow problem. The model layer knows tensor shapes, parallelism groups, routing decisions, and graph-capture constraints. The synthesizer knows topology and schedule structure. The programming layer knows which primitives and synchronization operations can represent a schedule. The runtime knows registered memory, queue pairs, congestion, failures, and transport capabilities. Today, much of this information is trapped in separate components. A future stack should pass enough information across boundaries to optimize globally while still maintaining clean abstractions.

One concrete cross-layer path is SyCCL-to-MSCCL++. SyCCL can generate a topology-specific schedule, but that schedule needs to be represented in a communication library. MSCCL++'s DSL is a natural target because it describes communication algorithms at a higher level and lowers them to primitives. The primitive layer then chooses channels that match hardware transfer modes. This would connect automatic synthesis to portable execution.

Another cross-layer path is MSCCL++-to-NCCLX. MSCCL++ exposes programmable algorithms, but production use requires initialization, resource management, transport backends, and fault localization. NCCLX already provides a production-oriented layer under PyTorch. A future system could allow MSCCL++-style kernels or DSL outputs to run inside an NCCLX-like runtime, gaining both programmability and operability.

A third path is NCCLX-to-SyCCL. NCCLX already supports topology-based optimization and custom algorithms through CTran, but the paper does not center automatic schedule synthesis. SyCCL could supply schedules for standard collectives or parallelism groups, while NCCLX handles deployment, registration, and failure handling. This would make the production framework more topology-adaptive.

6.2 Open Challenges

The first open challenge is a unified communication intermediate representation. Such an IR would need to describe collectives, point-to-point operations, RMA, synchronization, topology constraints, and GPU-resident metadata. It could serve as a target for model compilers, SyCCL-like synthesizers, and MSCCL++-like DSLs.

Designing such an IR is difficult because it must sit between very different levels of abstraction. If it is too high-level, it will look like existing collective APIs and fail to represent custom schedules or GPU-resident metadata. If it is too low-level, it will become a transport-specific assembly language that model compilers cannot target. A useful IR would likely need multiple layers: a semantic layer for collective intent, a schedule layer for ordering and chunk movement, and an execution layer for primitives and synchronization. This mirrors the thesis's main argument that communication should be viewed as a stack.

The second challenge is correctness and debuggability. Primitive communication APIs are powerful but can introduce subtle bugs in memory ordering and synchronization. Schedule synthesis can generate complex data movement that is difficult to inspect. NCCLX's Fault Analyzer shows one production response, but more general validation tools are needed.

The third challenge is adaptive runtime selection. The best communication path may change by message size, model phase, topology, and fault state. A runtime may need to choose between NCCL-style collectives, MSCCL++ kernels, SyCCL schedules, zero-copy paths, and GPU-resident operations. Making these decisions automatically remains open.

The fourth challenge is reliability-aware synthesis. SyCCL optimizes schedule performance, but production systems also care about fragile links, failure domains, and diagnosis cost. Integrating schedule synthesis with NCCLX-like operational signals would make communication optimization more robust.

The fifth challenge is memory registration and lifetime management. Zero-copy communication is attractive, but real frameworks allocate and free tensors dynamically. NCCLX addresses this through PyTorch allocator co-design, but a general solution across frameworks and hardware remains difficult. A communication IR or runtime API should include buffer lifetime and registration information so that zero-copy paths can be selected safely.

The sixth challenge is dynamic MoE communication. AllToAllvDynamic shows that GPU-resident metadata can reduce padding and CPU overhead, but dynamic routing also complicates scheduling and graph capture. Future systems must support communication whose size and destinations are known only after GPU computation. This challenge connects programming models, compiler support, runtime metadata, and transport execution.

The seventh challenge is performance portability. MSCCL++ shows encouraging portability results, but every new hardware feature may require a new channel or primitive extension. The system must decide which hardware features deserve first-class abstractions and which should remain backend-specific. Too many abstractions make the API unstable; too few abstractions prevent optimization.

An eighth challenge is evaluation standardization. Communication papers often report different metrics: algorithmic bandwidth, bus bandwidth, end-to-end latency, synthesis time, initialization time, CPU usage, memory overhead, and failure-localization time. All of these are valid, but without a common evaluation vocabulary it is difficult to compare systems. A future benchmark suite for AI communication should include static collectives, dynamic MoE communication, graph-captured inference, multi-dimensional parallelism, large-scale initialization, and failure scenarios. It should also separate small-message latency, large-message bandwidth, resource consumption, and operational recovery. This would make it easier to judge whether a new communication mechanism improves the whole system or only one narrow microbenchmark.

6.3 Summary

This chapter synthesized the three selected papers into a single systems stack. MSCCL++ contributes the programming layer, SyCCL contributes the schedule and evaluation layer, and

NCCLX contributes the runtime and production layer. The main open problem is not merely making one collective faster, but defining stable interfaces between these layers so that future AI systems can optimize communication automatically and operate reliably at scale.

The synthesis also clarifies why the three papers should be read together. MSCCL++ without a schedule synthesizer still requires experts to design algorithms. SyCCL without a programmable execution substrate still needs a way to deploy schedules. NCCLX without programmable synthesis may still require manual custom communication work. The future communication stack is likely to borrow from all three.

The broader lesson is that AI communication is converging with classical systems design. The field now needs abstractions, intermediate representations, scheduling algorithms, runtime resource management, debugging tools, and evaluation methodology. In other words, collective communication has become a system in the same sense as a compiler, database, or operating system: its value comes from the coherence of its layers.

One way to make the connection concrete is to imagine a training job being compiled for a new cluster. The compiler or runtime first identifies parallelism groups and communication operations. It asks a schedule synthesizer to generate candidate schedules for regular collectives, using topology information and message sizes. The chosen schedules are represented in a DSL. The DSL is lowered into primitive operations, where channels select the appropriate transport mode. The production runtime registers buffers, initializes communicators, monitors progress, and records traces for debugging. Every paper in this report contributes to one part of that flow.

Another way is to consider MoE inference. The router kernel produces GPU-resident metadata. A communication programming layer must allow this metadata to influence the operation. The runtime must avoid CPU preparation overhead and padded transfers. The schedule or transport layer must handle small, dynamic, possibly imbalanced messages. The observability layer must diagnose stalls. This example cuts across all three layers and shows why isolated APIs are not enough.

The open challenges therefore are interface challenges. What is the right IR for communication? What information should the model runtime expose to the communication layer? How should a synthesizer express schedules? How can a production runtime validate and debug them? How can hardware-specific features be exposed without destroying portability? These questions remain open precisely because the layers are only beginning to be connected.

The most important cross-layer interface is likely between the model runtime and the communication runtime. The model runtime knows which tensors will be communicated, which parallelism group owns them, whether the operation lies in prefill or decode, whether graph capture is active, and whether routing metadata is produced on the GPU. The communication runtime knows which buffers are registered, which transports are available, which links are congested, and which algorithms are implemented. Today, much of this knowledge is implicit. A future interface should make it explicit enough for automatic selection without exposing so much detail that model code becomes hardware-specific.

A second interface is between schedule synthesis and execution. SyCCL can generate schedules, but a schedule must eventually become executable operations. That requires a representation of chunking, send/receive or put operations, reductions, dependencies, synchronization, and possibly channel choices. MSCCL++'s DSL is a candidate, but a full integration would need additional metadata: topology assumptions, cost-model parameters, fallback algorithms, and validation checks. The goal should not be to make the synthesizer emit backend-specific code directly. Instead, it should emit a portable schedule representation that the runtime can lower using local hardware knowledge.

A third interface is between observability and optimization. NCCLX uses traces to diagnose faults, but traces could also feed performance adaptation. If the runtime observes congestion on certain links, repeated registration spikes, overloaded proxy threads, or frequent fallback to padded transfers, it could change algorithm selection or request new schedules. This would make communication optimization feedback-driven. The challenge is to avoid making the runtime unstable: adapting too aggressively can introduce unpredictability, while adapting too slowly misses performance opportunities.

The open challenges also have a human factor. Programmable communication gives experts more power, but it also raises the cost of understanding the system. A future stack will need documentation, validation, profiling, and visualization tools. A developer should be able to answer questions such as: Which collective is on the critical path? Which schedule was selected? Which channel executed it? Did it use zero-copy? How many queue pairs did it consume? Was routing metadata on the GPU or CPU? Which link or rank caused a stall? Without such tools, the stack may become too complex for practical use even if it is theoretically powerful.

The synthesis chapter therefore argues for disciplined layering rather than unlimited exposure. The goal is not to expose every low-level detail to every user. The goal is to expose the right information at the right boundary. Applications should express communication intent and provide useful metadata. Synthesizers should express schedule structure. Programming layers should express executable primitives and synchronization. Runtimes should manage resources and observability. Hardware backends should expose capabilities through stable abstractions. This division of labor is what makes a complex communication stack maintainable.

In the long run, collective communication may become a compiler/runtime co-optimization problem. The compiler can know tensor shapes, static parallelism groups, and repeated communication patterns. The runtime can know actual topology, registration state, and runtime congestion. A synthesizer can search schedules using both sources of information. A programmable communication layer can execute the result. This is not fully realized in any of the three selected papers, but each paper contributes a necessary part: MSCCL++ provides a programmable target, SyCCL provides synthesis, and NCCLX provides production execution and observability.

The final cross-layer lesson is that the communication stack should be judged by how well information flows through it. If topology information cannot reach the algorithm, schedules will be poor. If model-phase information cannot reach the runtime, inference and training may use the wrong path. If transport telemetry cannot reach the selector, the system cannot adapt to congestion or failures. If schedule structure cannot reach the debugger, failures will be opaque. Future communication systems should therefore be designed not only around fast data movement, but around controlled information movement across layers.

This information-flow perspective also changes how we think about abstraction. Traditional abstraction hides details. Systems abstraction should hide irrelevant details while preserving useful information. For collective communication, hiding every hardware detail from the algorithm may make optimization impossible, but exposing every hardware detail to the model developer is also unacceptable. The right abstraction must compress information. A topology abstraction can expose groups, dimensions, and bandwidth ratios without exposing every switch. A schedule abstraction can expose stages and dependencies without exposing every packet. A runtime abstraction can expose registration state and graph-capture constraints without exposing driver internals. Good abstraction is therefore selective visibility.

The same idea applies to portability. Portability does not mean every platform uses the same algorithm. It means the same semantic program can be lowered appropriately on different platforms. On one platform, a schedule may use SwitchChannel and switch-assisted aggregation. On another, it may use MemoryChannel or PortChannel. On one cluster, SyCCL may choose a schedule that exploits a strong intra-node fabric. On another, it may choose a different hierarchy. On one inference workload, NCCLX may use GPU-resident metadata. On another, a standard host-initiated collective may be sufficient. The stack is portable if these choices happen below a stable semantic interface.

The cross-layer synthesis also implies that communication optimization should be iterative. Profiling identifies bottlenecks. The programming layer provides possible implementations. The synthesizer searches schedules. The runtime executes and records traces. The evaluation layer measures model impact. The next iteration updates the schedule or policy. This loop is currently manual in many systems. A mature stack would automate more of it, turning communication optimization into a feedback process similar to compiler optimization or query planning.

One risk is that the combined stack becomes too complex. NCCLX, MSCCL++, and SyCCL each introduce nontrivial mechanisms. Combining all of them without discipline could create a

system that is powerful but difficult to reason about. This is why stable interfaces are essential. The programming layer should not depend on every detail of the synthesizer. The synthesizer should not depend on every detail of the transport backend. The runtime should not expose every low-level event to applications. Each boundary should pass the information needed for optimization and debugging, but no more. This is the engineering challenge behind the conceptual synthesis.

Another risk is that automatic systems make poor choices silently. A runtime selector may choose a schedule that looks good under an inaccurate cost model. A synthesizer may generate a schedule that performs well in simulation but poorly on real hardware. A zero-copy path may save copies but increase registration overhead. A GPU-resident path may reduce CPU overhead but complicate graph capture or synchronization. For this reason, future systems need not only automatic selection but also explainability: why was this algorithm chosen, what assumptions were used, and what fallback exists if those assumptions fail?

The cross-layer chapter therefore does not argue for a single monolithic framework. It argues for interoperability among layers. A production runtime should be able to execute standard collectives, hand-written primitive kernels, DSL-generated programs, and synthesized schedules. A programming layer should be able to target multiple transports. A synthesizer should be able to consume topology and cost information from different runtimes. A tracing layer should be able to relate low-level events back to high-level operations. This modular vision is more realistic than expecting one system to replace all existing communication infrastructure at once.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This report followed the same seven-chapter structure as the reference thesis while replacing its content with a systems study of collective communication for large-scale AI. It presented three representative papers: NCCLX, MSCCL++, and SyCCL. NCCLX shows that collective communication must be engineered as production infrastructure for 100K+ GPU training and inference. MSCCL++ shows that a layered programming model can support both portability and custom optimization. SyCCL shows that exploiting topology and collective symmetry can make schedule synthesis practical.

Across the report, I argued that collective communication is becoming a programmable systems substrate. It must remain simple for ordinary users, but expose enough structure for expert optimization. It must provide high bandwidth for training, low latency for inference, topology-aware schedules for clusters, and operational support for long-running jobs. The three selected papers each solve part of this broader problem.

This argument is consistent with the broader evolution of distributed machine learning. Horovod made distributed data-parallel training accessible through a relatively simple API [8]. Megatron-LM and ZeRO showed that large models require more structured parallelism and memory partitioning [5, 7]. GShard and Switch Transformer showed that sparse expert models introduce dynamic token exchange [4, 2]. NCCLX, MSCCL++, and SyCCL can be read as communication-system responses to these application trends: production runtime support, programmable communication interfaces, and topology-aware schedule synthesis.

The first contribution was to map the selected papers onto the reference thesis structure. Chapter 3 treated MSCCL++ as the programming-model layer, explaining primitives, channels, DSL execution, collective compatibility, and portability. Chapter 4 treated the workload-and-evaluation layer, explaining training, inference, topology, latency, bandwidth, scalability, and operability. Chapter 5 treated the runtime layer, explaining NCCLX CTran, DQPLB, zero-copy, AllToAllvDynamic, scalable initialization, Fault Analyzer, and SyCCL synthesis.

The second contribution was to preserve concrete paper details rather than only high-level summaries. The report includes MSCCL++ PortChannel request queues, MemoryChannel and SwitchChannel roles, one-phase and two-phase AllReduce algorithms, AMD and NVLink 4.0 portability effort, SyCCL sketches and sketch combinations, MILP sub-schedules, alpha-beta modeling, ASTRA-sim-based simulation, NCCLX tensor registration, DQPLB, GPU-resident collectives, and large-scale initialization.

The third contribution was to show that the reference thesis structure can be reused for a different systems topic. In the reference thesis, Chapter 3 is a programming model, Chapter 4 is workload and evaluation, and Chapter 5 is runtime. This report follows the same organization. The topic is different, but the systems logic is similar: a complex application domain requires abstractions, workloads, execution mechanisms, and cross-layer interfaces. This is why fully mimicking the structure is meaningful rather than cosmetic.

The fourth contribution was to identify a combined future direction. Instead of treating

NCCLX, MSCCL++, and SyCCL as separate solutions, the report argues that a complete future stack would combine production runtime support, programmable communication interfaces, and automatic schedule synthesis. The combined stack would be more useful than any single component alone.

7.2 Future Work

Future collective communication systems will likely combine the strengths of these approaches. One direction is to use a SyCCL-like synthesizer to generate schedules expressed through an MSCCL++-like DSL and executed inside an NCCLX-like production framework. Another direction is adaptive runtime selection, where the system chooses among collectives, custom kernels, zero-copy paths, GPU-resident metadata, and synthesized schedules based on message size, topology, and model phase.

Finally, future work should evaluate communication systems end to end. Microbenchmarks are necessary, but they are not sufficient. The most useful results connect mechanisms to training step time, inference latency, startup time, resource usage, and failure recovery. NCCLX, MSCCL++, and SyCCL all move in this direction, and their combination points toward a more complete communication stack for future AI systems.

In the longer term, the communication layer may become compiler-managed. A model compiler or distributed runtime could know tensor shapes, parallelism groups, expert routing constraints, and hardware topology. It could then emit a communication IR, call a SyCCL-like synthesizer, lower the schedule to MSCCL++ primitives, and execute it through an NCCLX-like transport runtime. This would make communication optimization less manual and more integrated with model execution.

Another future direction is reliability-aware and congestion-aware scheduling. Current schedule synthesis focuses mainly on performance, but production clusters have failures, noisy neighbors, and uneven link health. A future synthesizer could incorporate fault domains, congestion signals, and diagnosis cost. Such a system would treat reliability not as an afterthought but as part of the schedule objective.

Finally, future systems should expose better debugging interfaces for communication programs. As communication becomes programmable, bugs will move from library internals to user-defined schedules and kernels. Tools similar to NCCLX's CollTrace, combined with static validation for MSCCL++ DSL programs and simulation for SyCCL schedules, could make programmable communication safer to deploy.

A final future direction is standardization. Today, many organizations build custom communication layers because the standard APIs do not expose enough control. If every organization builds a separate stack, the ecosystem fragments. The challenge is to standardize the right level: not so high-level that optimization becomes impossible, and not so low-level that applications become hardware-specific. MSCCL++ primitives, SyCCL schedule descriptions, and NCCLX execution metadata all suggest possible ingredients for such a standard.

The central conclusion is therefore not that one paper solves collective communication. The conclusion is that modern collective communication requires a stack. The programming layer must express the operation. The workload layer must define what matters. The runtime layer must execute and diagnose it. The cross-layer interface must let optimization flow between these layers. NCCLX, MSCCL++, and SyCCL are representative examples of this transition.

For the purpose of this course report, this also explains why the final paper keeps the reference thesis structure rather than organizing the content as three independent paper summaries. A paper-by-paper summary would be simpler, but it would miss the systems relationship between the works. The reference thesis is organized around layers, and the same organization fits collective communication: MSCCL++ corresponds to the programming layer, the evaluation results and workload definitions correspond to the workload layer, NCCLX and SyCCL correspond to runtime and scheduling, and the final synthesis connects them. This makes the report closer in spirit to the provided example while still using the selected communication papers as the actual content.

The final takeaway is pragmatic. A future communication stack for AI should let ordinary users call familiar collectives, let experts define custom communication algorithms, let synthesizers generate topology-aware schedules, and let production runtimes execute those schedules with memory registration, congestion control, initialization, and fault diagnosis. If any one of these pieces is missing, the system either becomes too rigid, too manual, too hard to operate, or too difficult to port. The three selected papers together define the outline of such a stack.

This is also why the paper keeps both technical detail and structural imitation. The structure follows the reference thesis, but the substance remains tied to NCCLX, MSCCL++, and SyCCL. The goal is not only to look like the reference document, but to make the same kind of layered systems argument in a different technical domain.

The conclusion can be restated as a design principle: a future AI communication stack should be compatible by default, programmable when needed, synthesizable when possible, and observable in production. Compatibility preserves adoption and lets existing applications benefit from improvements. Programmability lets experts express algorithms that fixed libraries do not yet provide. Synthesis reduces the manual burden of designing topology-specific schedules. Observability makes the system deployable at scale. NCCLX, MSCCL++, and SyCCL each emphasize different parts of this principle, but the combined direction is clearer than any single paper alone.

The most immediate future work would be an integration prototype. Such a prototype could take a small set of SyCCL-generated schedules, represent them in an MSCCL++-like DSL, and execute them through a runtime that exposes NCCLX-like transport and tracing hooks. The goal would not be to reproduce all production features at once. It would be to test whether the interfaces align: Can a synthesized schedule be represented cleanly? Can the primitive layer express required synchronization? Can the runtime choose channels automatically? Can traces map runtime behavior back to schedule stages? Answering these questions would turn the synthesis in this report into an implementable research agenda.

A second future direction is dynamic MoE communication as a focused benchmark. MoE inference combines many hard problems in one workload: GPU-produced metadata, small messages, irregular counts, latency sensitivity, graph-capture constraints, and expert-load imbalance. A benchmark centered on MoE routing and expert exchange would test whether a communication stack can handle dynamic behavior rather than only static collectives. NCCLX's AllToAllvDynamic provides a production example, but the broader community would benefit from a standardized benchmark that compares padded host-driven AllToAllv, GPU-resident metadata paths, synthesized schedules, and primitive-level custom kernels.

A third direction is reliability-aware communication synthesis. Current synthesis work mainly optimizes performance under assumed topology and cost parameters. Production clusters, however, have degraded links, failed hosts, transient congestion, and maintenance domains. A reliability-aware synthesizer could avoid fragile links, prefer schedules with easier diagnosis, or generate fallback schedules for common failure modes. NCCLX's Fault Analyzer provides the observability side of this idea, while SyCCL provides the synthesis side. Combining them would make schedule selection not only faster, but safer.

A fourth direction is portable correctness validation. As communication becomes programmable, the system needs tools that can verify memory ordering, reduction correctness, dependency satisfaction, and buffer lifetime. These tools should operate at several levels: DSL static checks, primitive-level race detection, schedule simulation, and runtime trace validation. The challenge is to provide useful guarantees without making the programming model too restrictive. This is especially important for GPU-resident and one-sided operations, where bugs may appear only under specific timing conditions.

The final future direction is standardization of intermediate abstractions. The community does not need every organization to implement a separate private communication stack. At the same time, standardizing only high-level collectives is no longer enough. A useful standard might include a small set of primitive operations, a schedule representation, topology and cost metadata, and observability hooks. Such a standard would allow algorithm research, runtime engineering, and hardware support to interoperate. The three selected papers do not define this

standard, but they point to its ingredients.

Overall, the report shows that collective communication is becoming one of the central systems problems of large-scale AI. Model quality and hardware scale will continue to grow, but scaling will be limited if communication remains rigid, opaque, or manually specialized. The path forward is a layered stack that can express communication, understand workloads, synthesize schedules, execute efficiently, and diagnose failures. NCCLX, MSCCL++, and SyCCL are early examples of this direction. Their combination suggests that the next generation of AI communication systems will look less like a fixed library and more like a programmable, adaptive, production-grade runtime.

This conclusion also changes how future researchers should frame contributions in this area. A paper that proposes a faster collective should explain which workload regime it targets, which topology assumptions it makes, how it interacts with memory registration, whether it consumes SMs, and how it would be debugged in a larger stack. A paper that proposes a new programming interface should explain not only expressiveness, but also safety, validation, profiling, and compatibility. A paper that proposes a scheduler should explain how schedules are represented, how they are lowered, how cost models are calibrated, and when schedules are invalidated. A paper that proposes a production runtime should explain both data-path performance and control-path behavior. The selected papers are valuable because each moves beyond a narrow benchmark in at least one of these directions.

For course-report purposes, the main intellectual result is the layered mapping. NCCLX, MSCCL++, and SyCCL initially look like three separate systems papers. NCCLX emphasizes production scale, MSCCL++ emphasizes APIs and primitives, and SyCCL emphasizes scheduling. After reading them together, they form a single design space. MSCCL++ answers how communication can be expressed. SyCCL answers how communication can be planned. NCCLX answers how communication can be executed and diagnosed in production. The layered thesis structure makes this relationship explicit and therefore turns three paper summaries into one systems argument.

There are also limitations in this report. It does not implement an integrated stack, so the proposed NCCLX–MSCCL++–SyCCL combination remains conceptual. It relies on reported results from the selected papers rather than rerunning experiments on the same cluster. It does not provide a complete taxonomy of all GPU communication libraries or all schedule-synthesis systems. It also does not resolve difficult correctness questions for programmable communication. These limitations are acceptable for a thesis-style course report, but they point to future empirical work. The next step after this synthesis would be to build a small prototype that tests whether a synthesized schedule can really be lowered into a programmable primitive layer and traced by a production-style runtime.

Despite these limitations, the core claim is robust. The pressure on communication is coming from several independent directions: larger clusters, larger models, more complex parallelism, dynamic MoE routing, tighter inference latency targets, heterogeneous hardware, and higher reliability expectations. A single fixed collective library cannot absorb all of these pressures without becoming either too rigid or too complicated internally. Layering is the pragmatic answer. Stable high-level APIs preserve usability. Programmable lower layers support specialization. Synthesizers reduce manual schedule design. Production runtimes provide initialization, resource management, and fault diagnosis. This is the systems pattern that unifies the report.

The final practical takeaway for engineers is to avoid optimizing collective communication in isolation. Before selecting or designing an algorithm, one should ask where the operation sits in the model, whether it is latency-bound or bandwidth-bound, whether its metadata is static or GPU-produced, whether buffers can be registered and reused, whether the operation runs under graph capture, whether it contends with compute kernels, what topology it crosses, and how failures will be diagnosed. These questions are exactly the questions surfaced by NCCLX, MSCCL++, and SyCCL. They turn communication optimization from a library call into a cross-layer systems task.

The final takeaway for researchers is that there is room for new abstractions. The field needs a communication IR that can express semantic collectives, schedule structure, synchronization, and

topology assumptions. It needs cost models that connect simulator predictions to real hardware behavior. It needs validation tools for GPU-resident and one-sided communication programs. It needs benchmarks that include dynamic inference and operational failures. It needs runtimes that can select among standard collectives, programmable kernels, synthesized schedules, and GPU-resident paths. These are not small engineering details; they are the foundation for scaling future AI workloads.

In summary, NCCLX, MSCCL++, and SyCCL should be read as complementary steps toward a new communication stack. NCCLX expands the communication library into production infrastructure. MSCCL++ expands the programming surface into a hierarchy of collectives, DSLs, and primitives. SyCCL expands algorithm design into topology-aware synthesis. The next generation of collective communication systems will likely combine these ideas, because large-scale AI requires all of them at once: stable interfaces for adoption, flexible programming for innovation, automatic scheduling for topology, efficient transports for performance, and observability for operation.

This final synthesis also gives a concrete checklist for evaluating future work. First, a system should state its communication semantics clearly: what operation is being performed, what data dependencies exist, and what correctness conditions must hold. Second, it should state its workload assumptions: message size, topology, model phase, metadata location, and reuse frequency. Third, it should state its execution assumptions: whether communication is host-initiated, GPU-resident, device-initiated, staged, zero-copy, SM-using, or SM-free. Fourth, it should state its operational assumptions: how initialization works, how resources are bounded, how failures are diagnosed, and how the system falls back when an optimized path is unavailable. These questions are not overhead; they are the minimum context needed to judge whether a communication result will matter in a real AI stack.

The report also suggests a useful division between short-term and long-term progress. In the short term, the community can improve individual mechanisms: faster zero-copy paths, safer primitive APIs, better schedule search, lower CPU overhead, and more useful traces. These improvements are valuable even if they are deployed independently. In the long term, the larger gain will come from connecting mechanisms. A synthesizer that can target a portable DSL is more useful than one that produces an isolated schedule. A primitive API that can consume topology and runtime cost information is more useful than one that only exposes low-level operations. A production runtime that can execute synthesized and programmable communication is more useful than one that only contains hand-selected algorithms. The long-term direction is therefore integration through stable interfaces.

There is one final reason collective communication deserves this systems treatment: it sits at the boundary between expensive hardware and fast-changing models. Hardware vendors add new interconnect features, memory systems, and networking capabilities. Model builders change architectures, parallelism strategies, routing policies, and serving patterns. The communication layer is where these two rates of change meet. If the layer is too rigid, new models cannot exploit new hardware quickly. If it is too low-level, every model team must become a communication-systems team. The right stack absorbs hardware change through channels and primitives, absorbs model change through expressive APIs and metadata, and uses synthesis and runtime policy to connect them.

Thus, the final answer to the report's three research questions is unified. Collective communication should be programmed through layered interfaces, because different users need different levels of control. It should be evaluated through multi-phase workloads, because training, prefill, decoding, MoE routing, initialization, and recovery stress different parts of the system. It should be executed through adaptive runtimes and topology-aware schedules, because no single algorithm or transport path is optimal across scales and phases. NCCLX, MSCCL++, and SyCCL do not merely provide three isolated optimizations; together they define the outline of a practical communication systems stack for future large-scale AI.

If this report were extended into a full research project, the most useful next artifact would be a small but end-to-end experimental stack. It would not need to match the scale of Meta's production environment or Alibaba's full synthesis evaluation. Instead, it could choose a manageable

cluster, define several representative collectives and MoE-style dynamic exchanges, generate a few topology-aware schedules, lower them into a programmable DSL, and execute them through a runtime that records cross-layer traces. The purpose would be to test the interfaces: whether workload metadata can guide schedule selection, whether synthesized schedules can be represented cleanly, whether primitive execution preserves correctness, and whether traces explain performance and failures. Such a prototype would make the thesis's central claim measurable. If the interfaces are clean, the stack should support new schedules and transports without rewriting applications. If the interfaces are poor, integration effort will dominate any performance gain. This kind of end-to-end validation is the natural continuation of the systems argument developed throughout the report.

The value of such a prototype would not be limited to speedup. It would also reveal which abstractions are too weak, which metadata is missing, which runtime decisions should be automatic, and which debugging views users actually need. That evidence would make future communication-system design more concrete than isolated microbenchmarks or purely conceptual stack diagrams.

Most importantly, it would test whether the proposed layered design remains understandable under implementation pressure, where clean ideas must coexist with hardware constraints, framework constraints, deployment constraints, and the practical need for safe fallback behavior.

That implementation pressure is the real test of any systems abstraction for collective communication.

Bibliography

[1] Jiamin Cao, Shangfeng Shi, Jiaqi Gao, et al. SyCCL: Exploiting Symmetry for Efficient Collective Communication Scheduling. In *Proc. ACM SIGCOMM*, 2025.

[2] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[3] Khaled Hamidouche, John Bachan, Pak Markthub, et al. GPU-Initiated Networking for NCCL. *arXiv preprint arXiv:2511.15076*, 2025.

[4] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *Proc. ICLR*, 2021.

[5] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, et al. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proc. SC*, 2021.

[6] NVIDIA Corporation. NVIDIA Collective Communication Library (NCCL). Technical documentation, 2025.

[7] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *Proc. SC*, 2020.

[8] Alexander Sergeev and Mike Del Balso. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv preprint arXiv:1802.05799*, 2018.

[9] Aashaka Shah, Abhinav Jangda, Binyang Li, et al. MSCCL++: Rethinking GPU Communication Abstractions for Cutting-Edge AI Applications. *arXiv preprint arXiv:2504.09014*, 2025.

[10] Min Si, Pavan Balaji, Yongzhou Chen, et al. Collective Communication for 100k+ GPUs. *arXiv preprint arXiv:2510.20171*, 2025.

[11] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. Optimization of Collective Communication Operations in MPICH. *International Journal of High Performance Computing Applications*, 19(1):49–66, 2005.